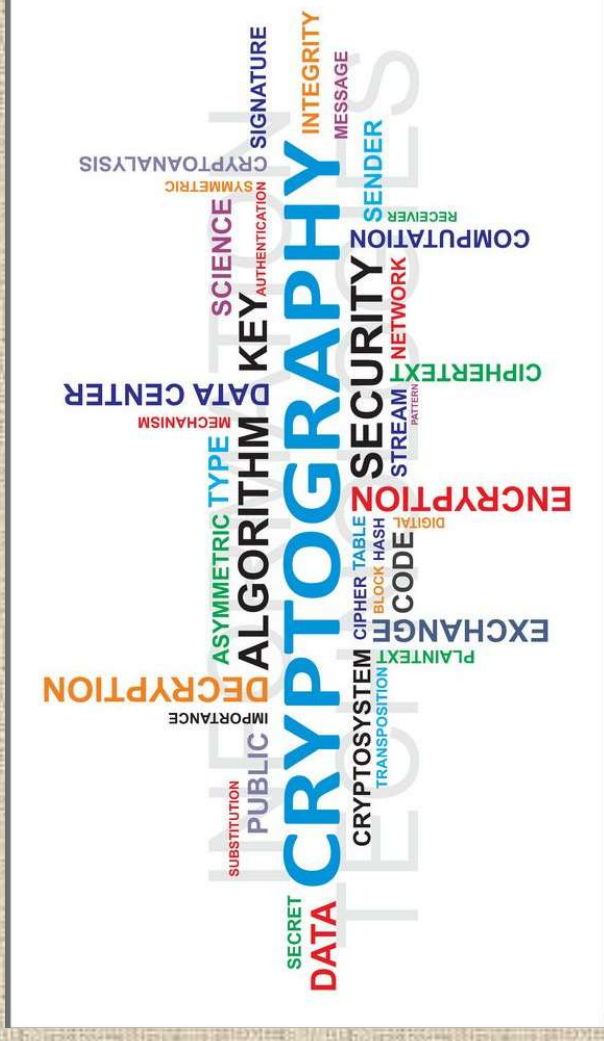
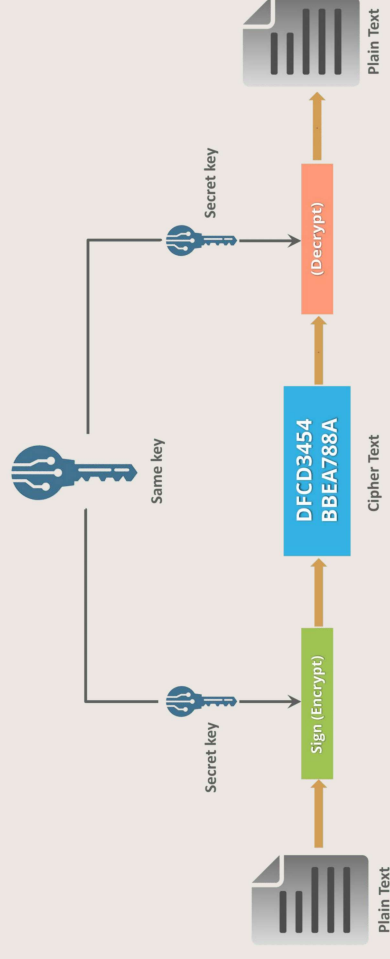


Modern Symmetric Key Cryptography



Modern Symmetric Key Cryptography



Symmetric cryptography is a class of algorithms for cryptography that use the same keys for both encryption of plaintext and decryption of cipher text. Examples are AES, DES.

Two branches of the symmetric key crypto family tree: **stream ciphers** and **block ciphers**

- **Stream ciphers** perform “bit-by-bit” crypto operations.
- A **block cipher** breaks the input into fixed-size blocks and then progresses the crypto operations.

Stream Ciphers

- A **stream cipher** is a symmetric cipher where convert one symbol of Plaintext immediately into a symbol of Ciphertext.
- A key K of n bits in length
- Keystream Generator(K) = S ,

Now, given a keystream $S = s_0, s_1, s_2 \dots$, and plaintext $P = p_0, p_1, p_2 \dots$ we generate the ciphertext $C = c_0, c_1, c_2 \dots$ by XOR-ing the corresponding bits, that is,

$$c_0 = p_0 \oplus s_0, \quad c_1 = p_1 \oplus s_1, \quad c_2 = p_2 \oplus s_2, \dots$$

To decrypt ciphertext C , the keystream S is again used, that is,

$$p_0 = c_0 \oplus s_0, \quad p_1 = c_1 \oplus s_1, \quad p_2 = c_2 \oplus s_2, \dots$$

Stream Ciphers: Examples

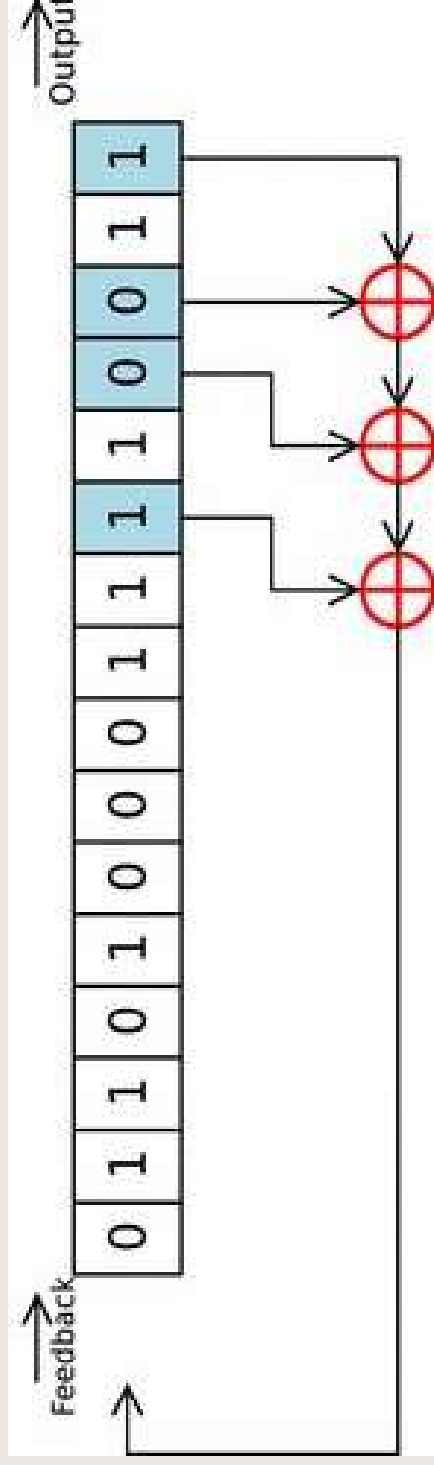
- Two stream cipher algorithms, A5/1 and RC4, both of which have been widely deployed.
- A5/1
 - Based on shift registers
 - Used in GSM mobile phone system
- RC4
 - Based on a changing lookup table
 - Used many places

Stream Cipher - Types

1. In a [synchronous stream cipher](#), a stream of pseudo-random digits is generated independently of the plaintext and ciphertext messages, and then combined with the plaintext (to encrypt) or the ciphertext (to decrypt).
2. [Asynchronous stream ciphers](#) use several of the previous N ciphertext digits to compute the keystream. (e.g., Cipher-Feedback Mode)

Example: linear feedback shift registers (LFSR)

- A linear feedback shift register (LFSR) linear feedback shift register (LFSR) is a shift register whose input bit is the output of a linear function of two or more of its previous states (taps)
- Example : $x^{16} + x^{14} + x^{13} + x^{11} + 1$ corresponds to LFSR of length 16



- The LFSR generates a sequence of length $2^{16} - 1 = 65535$ bits until it reaches its original internal state again.

Stream Cipher – Example (1): A5/1

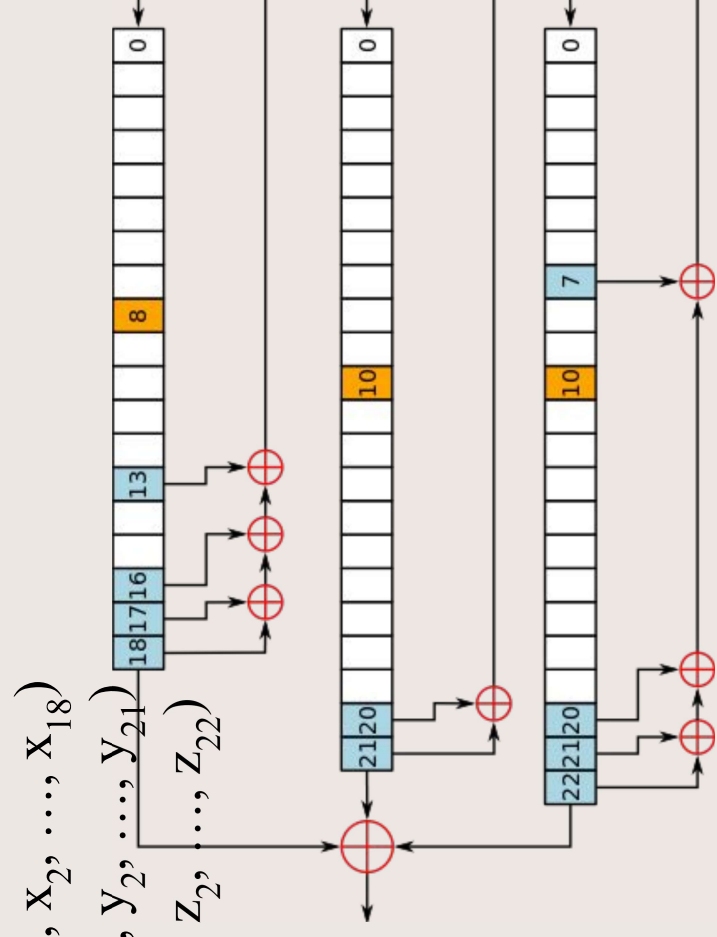
- A5/1 is a stream cipher used to provide over-the-air communication privacy in the GSM cellular telephone standard.
- A5/1 consists of 3 *shift registers* X, Y and Z

1. A5/1 initialization:

1. Register X hold 19 bits of the key ($x_0, x_1, x_2, \dots, x_{18}$)
2. Register Y hold 22 bits of the key ($y_0, y_1, y_2, \dots, y_{21}$)
3. Register Z hold 23 bits of the key ($z_0, z_1, z_2, \dots, z_{22}$)

2. A5/1 key stream bit algorithm:

1. $m = \text{maj}(x_8, y_{10}, z_{10})$
2. if ($x_8 = m$) then *X steps*
3. if ($y_{10} = m$) then *Y steps*
4. if ($z_{10} = m$) then *Z steps*
5. $s = x_{18} \oplus y_{21} \oplus z_{22}$
6. Go to step 1



Stream Cipher: A5/1

- When register X steps , the following occur

- $t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$
- $x_i = x_{i-1}$ for $i = 18, 17, 16, \dots, 1$
- $x_0 = t$

- for registers Y and Z , each step consists of

- $t = y_{20} \oplus y_{21}$
- $y_i = y_{i-1}$ for $i = 21, 20, 19, \dots, 1$
- $y_0 = t$

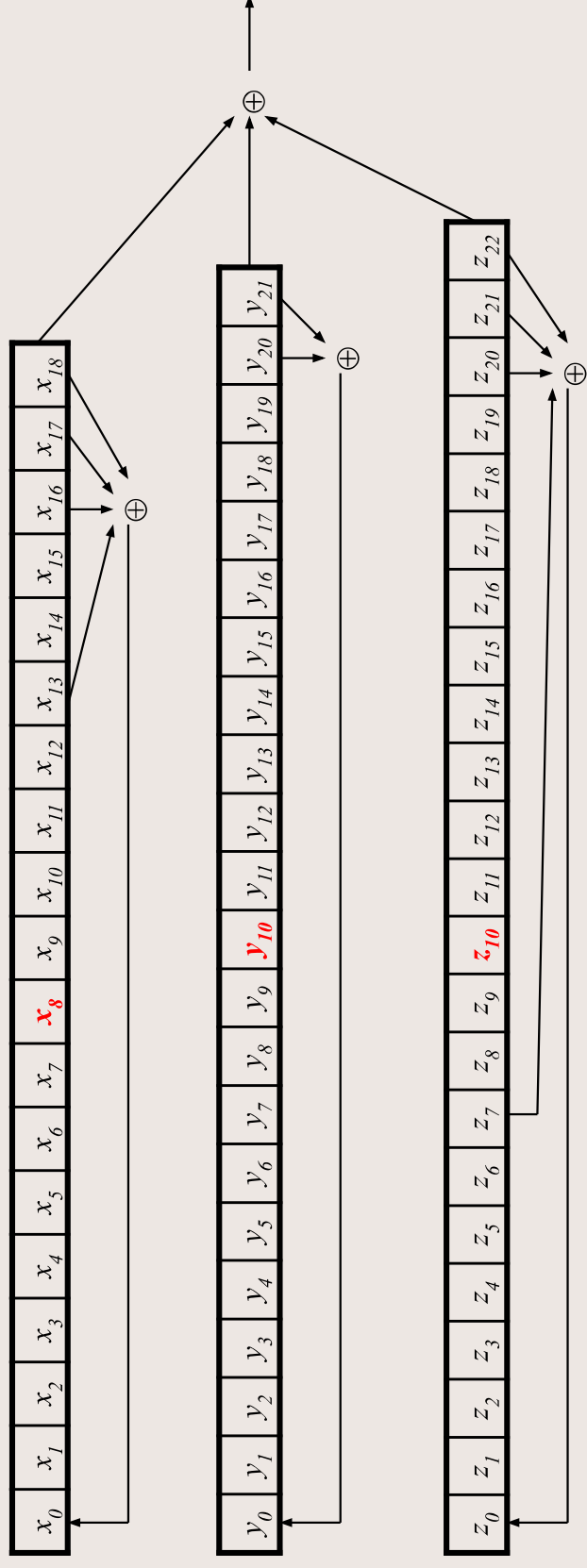
- and

- $t = z_7 \oplus z_{20} \oplus z_{21} \oplus z_{22}$
- $z_i = z_{i-1}$ for $i = 22, 21, 20, \dots, 1$
- $z_0 = t$

The majority vote function: $\text{maj}(x, y, z)$ return 0 if the majority of x, y, and z are 0, otherwise it returns 1.

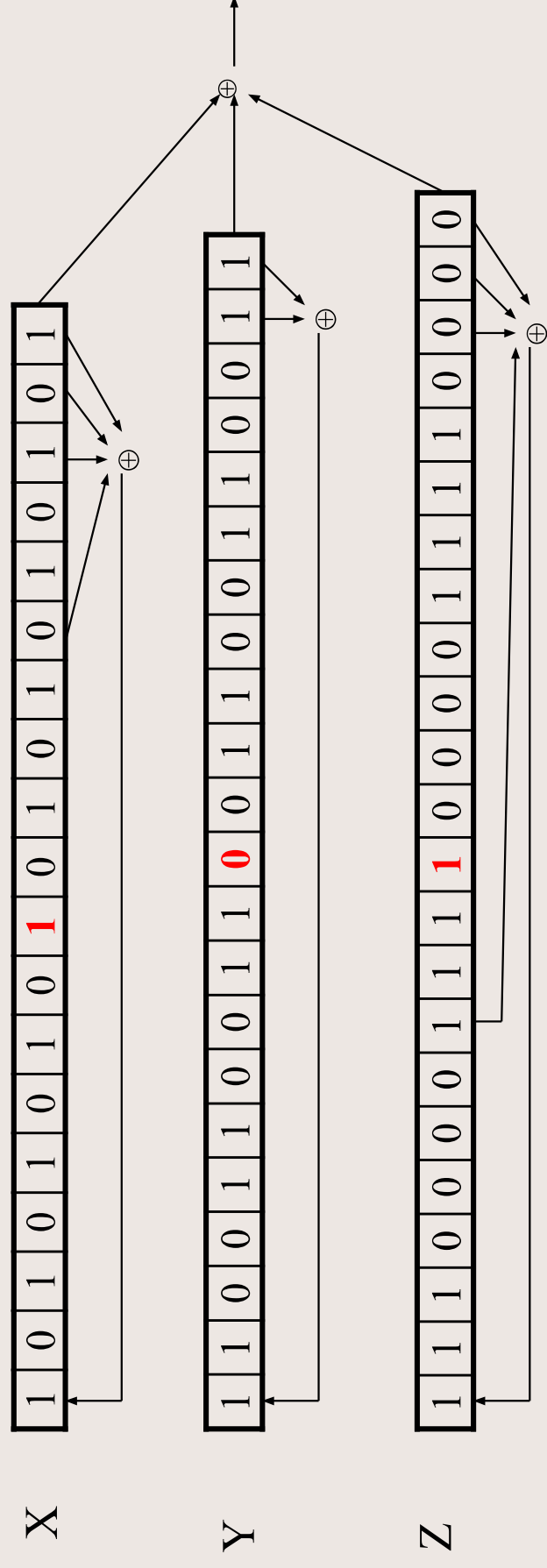
- Examples: $\text{maj}(0,1,0) = 0$ and $\text{maj}(1,1,0) = 1$

Stream Cipher: A5/1



- Each value is a single bit
- Key is used as initial fill of registers
- Each register steps or not, based on (x_8, y_{10}, z_{10})
- Keystream bit is XOR of right bits of registers

Stream Cipher: A5/1



- In this example, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(\mathbf{1}, \mathbf{0}, \mathbf{1}) = \mathbf{1}$
- Register X steps, Y does not step, and Z steps
- Keystream bit is XOR of right bits of registers
- Here, keystream bit will be $0 \oplus 1 \oplus 0 = 1$

Stream Cipher – Example (2): RC4

- RC4 means Rivest Cipher 4 invented by Ron Rivest in 1987 for RSA Security.
- It is a Stream Ciphers. Stream Ciphers operate on a stream of data byte by byte.
- RC4 stream cipher is one of the most widely used stream ciphers because of its simplicity and speed of operation.
- It is a variable key-size stream cipher with byte-oriented operations. It uses either 64 bit or 128-bit key sizes.
- It is generally used in applications such as Secure Socket Layer (SSL), Transport Layer Security (TLS), and also used in IEEE 802.11 wireless LAN std.

RC4 Algorithm

- The key as $\text{key}[i]$, for $i = 0, 1, \dots, N - 1$, where each $\text{key}[i]$ is a byte.
- The lookup table as $S[i]$, where each $S[i]$ is also a byte.

Step 1: initializes the lookup table using the key

```
for i = 0 to 255
    S[i] = i
    K[i] = key [ i mod N ]
next i
j = 0
for i = 0 to 255
    j = ( j + S[i] + K[i] ) mod 256
    swap ( S[i] , S[j] )
next i
```

RC4 Algorithm

Step2: Generates keystream byte:

$i = j = 0$

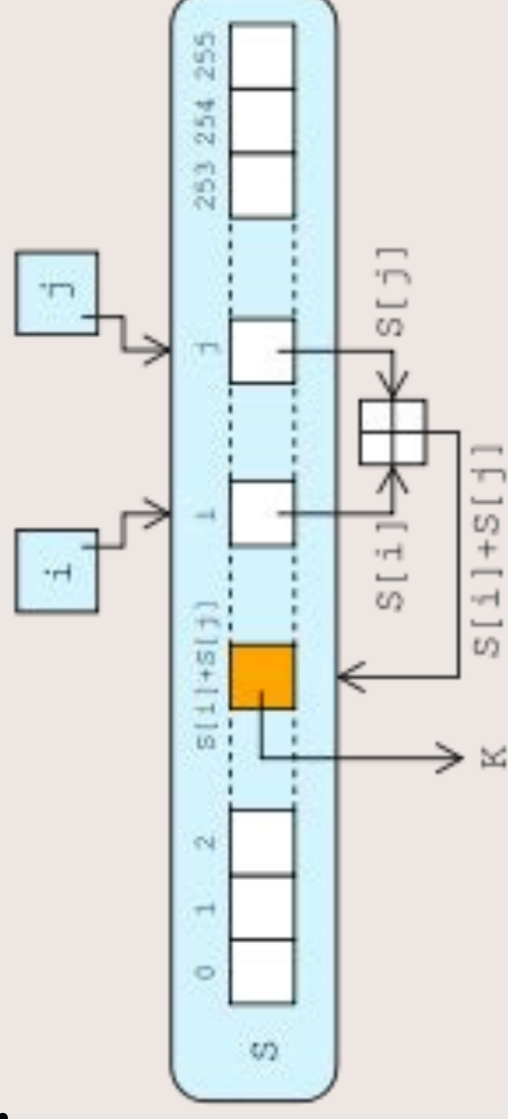
$i = (i + 1) \bmod 256$

$j = (j + S[i]) \bmod 256$

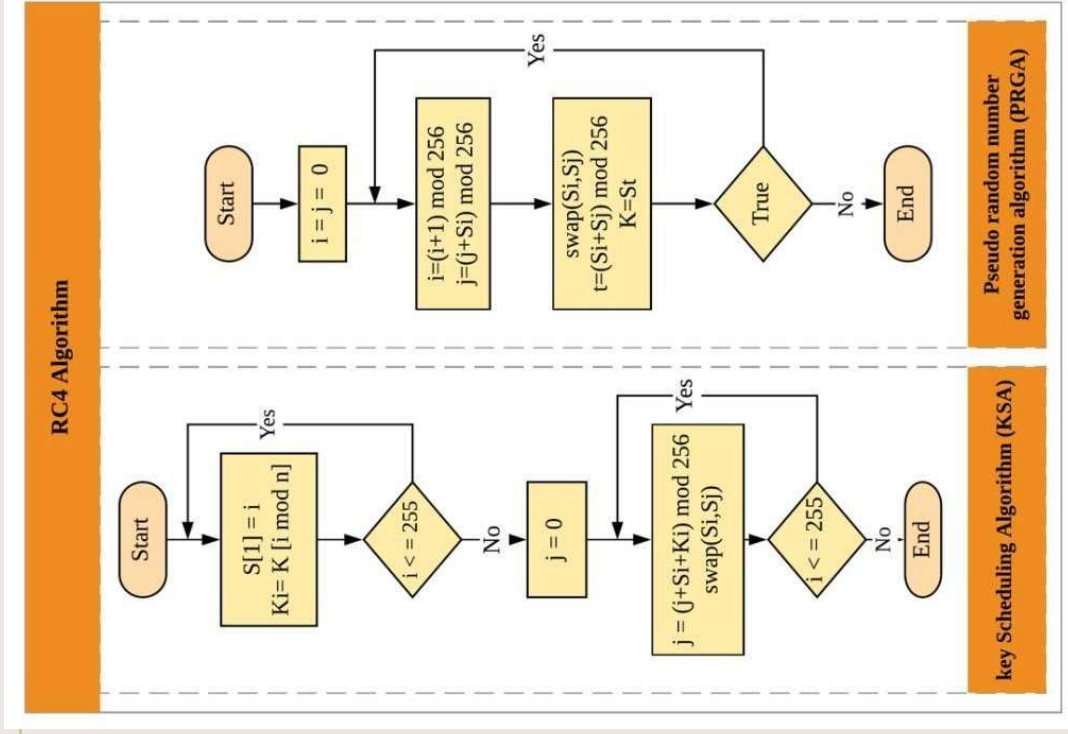
swap ($S[i]$, $S[j]$)

$t = (S[i] + S[j]) \bmod 256$

keystreamByte = $S[t]$

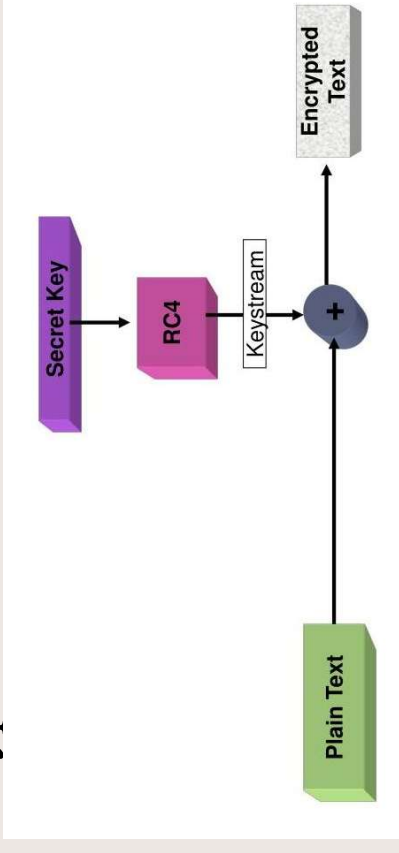


RC4 Algorithm



RC4-Encryption and Decryption

- **Encryption:** This keystream is now XOR with the plain text, this XORing is done byte by byte to produce the encrypted text.



- **Decryption** is achieved by doing the same byte-wise X-OR operation on the Ciphertext.

RC4: Example

Assume we use a 4 x 3-bit key, **K**, and plaintext **P** as below:

K = [1 2 3 6]
P = [1 2 2 2]

The first step is to generate the stream.

Initialise the state vector **S** and temporary vector **T**. **S** is initialised so the **S**[*i*] = *i*, and **T** is initialised so it is the key **K** (repeated as necessary).

S = [0 1 2 3 4 5 6 7]
T = [1 2 3 6 1 2 3 6]

Now perform the initial permutation on **S**.

```
j = 0;
for i = 0 to 7 do
    j = (j + S[i] + T[i]) mod 8
    Swap(S[i],S[j]);
end
```

We will step through for each iteration of *i*:

```
For i = 0:
j = (0 + 0 + 1) mod 8
  = 1
Swap(S[0],S[1]);
```

So in the 1st iteration **S**[0] must be swapped with **S**[1] giving:

S = [1 0 2 3 4 5 6 7]

Table 3.1: RC4 Initialization

```
for i = 0 to 255
    S[i] = i
    K[i] = key[i mod N]
next i
j = 0
for i = 0 to 255
    j = (j + S[i] + K[i]) mod 256
    swap(S[i],S[j])
next i
i = j = 0
```

RC4: Example

The results of the remaining 7 iterations are:

```
For i = 1:
  j = 3
  Swap(S[1],S[3])
  S = [1 3 2 0 4 5 6 7];

For i = 2:
  j = 0
  Swap(S[2],S[0]);
  S = [2 3 1 0 4 5 6 7];

For i = 3:
  j = 6;
```

```
Swap(S[3],S[6])
S = [2 3 1 6 4 5 0 7];

For i = 4:
  j = 3
  Swap(S[4],S[3])
  S = [2 3 1 4 6 5 0 7];

For i = 5:
  j = 2
  Swap(S[5],S[2]);
  S = [2 3 5 4 6 1 0 7];

For i = 6:
  j = 5;
  Swap(S[6],S[5])
  S = [2 3 5 4 6 0 1 7];

For i = 7:
  j = 2;
  Swap(S[7],S[2])
  S = [2 3 7 4 6 0 1 5];

Hence, our initial permutation of S gives:
S= [2 3 7 4 6 0 1 5];
```

RC4: Example

Hence, our initial permutation of **S** gives:

S= [2 3 7 4 6 0 1 5];

Now we generate 3-bits at a time, *k*, that we XOR with each 3-bits of plaintext to produce the ciphertext. The 3-bits *k* is generated by:

```
i, j = 0;
while (true) {
    i = (i + 1) mod 8;
    j = (j + S[i]) mod 8;
    Swap (S[i], S[j]);
    t = (S[i] + S[j]) mod 8;
    k = S[t];
}
```

The first iteration:

```
S = [2 3 7 4 6 0 1 5]
i = (0 + 1) mod 8 = 1
j = (0 + S[1]) mod 8 = 3
Swap(S[1],S[3])
S = [2 4 7 3 6 0 1 5]
t = (S[1] + S[3]) mod 8 = 7
k = S[7] = 5
```

Remember, that **P** is:

P = [1 2 2 2]

So our first 3-bits of ciphertext is obtained by: *k* XOR *P*₁

5 XOR 1 = 101 XOR 001 = 100 = 4

The second iteration:

```
S = [2 4 7 3 6 0 1 5]
i = (1 + 1 ) mod 8 = 2
j = (3 + S[2]) mod 8 = 2
```

```
Swap(S[2],S[2])
S = [2 4 7 3 6 0 1 5]
t = (S[2] + S[2]) mod 8 = 6
k = S[6] = 1
```

Second 3-bits of ciphertext are:

1 XOR 2 = 001 XOR 010 = 011 = 3

The third iteration:

```
S = [2 4 7 3 6 0 1 5]
i = (2 + 1) mod 8 = 3
j = (2 + S[3]) mod 8 = 5
Swap(S[3],S[5])
S = [2 4 7 0 6 3 1 5]
t = (S[3] + S[5]) mod 8 = 3
k = S[3] = 0
```

Third 3-bits of ciphertext are:

0 XOR 2 = 000 XOR 010 = 010 = 2

RC4: Example

The final iteration:

```
S = [2 4 7 0 6 3 1 5]
i = (1 + 3 ) mod 8 = 4
j = (5 + S[4]) mod 8 = 3
Swap(S[4],S[3])
S = [2 4 7 6 0 3 1 5]
t = (S[4] + S[3]) mod 8 = 6
k = S[6] = 1

    Last 3-bits of ciphertext are:
1 XOR 2 = 001 XOR 010 = 011 = 3
```

So to encrypt the plaintext stream **P** with key **K**

get **C**:

```
P = [1 2 2 2]
K = [5 1 0 1]
C = [4 3 2 3]
```

Or in binary:

```
P = 001010010010
K = 101001000001
C = 100011010011
```

Advantages and Disadvantages of RC4

- **Advantages**
- RC4 stream ciphers are simple to use.
- The speed of operation in RC4 is fast as compared to other ciphers.
- RC4 stream ciphers are strong in coding and easy to implement.
- RC4 stream ciphers do not require more memory.
- RC4 stream ciphers are implemented on large streams of data.
- **Disadvantages**
- If RC4 is not used with strong MAC then encryption is vulnerable to a bit-flipping attack.
- RC4 stream ciphers do not provide authentication.
- RC4 algorithm requires additional analysis before including new systems.
- RC4 stream ciphers cannot be implemented on small streams of data.
- RC4 fails to discard the beginning of output keystream or fails to use non-random or related keys for the algorithm.

Review Questions

- 1- find the key stream then encrypt the letter A by using A5/1 method , where the letter is given in ASCII code. The ASCII value for A is $65_{10} = 1000001_2$

0	1	1	1	0	1	1	0	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

X

1	0	0	1	1	0	1	0	1	0	0	1	1	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Y

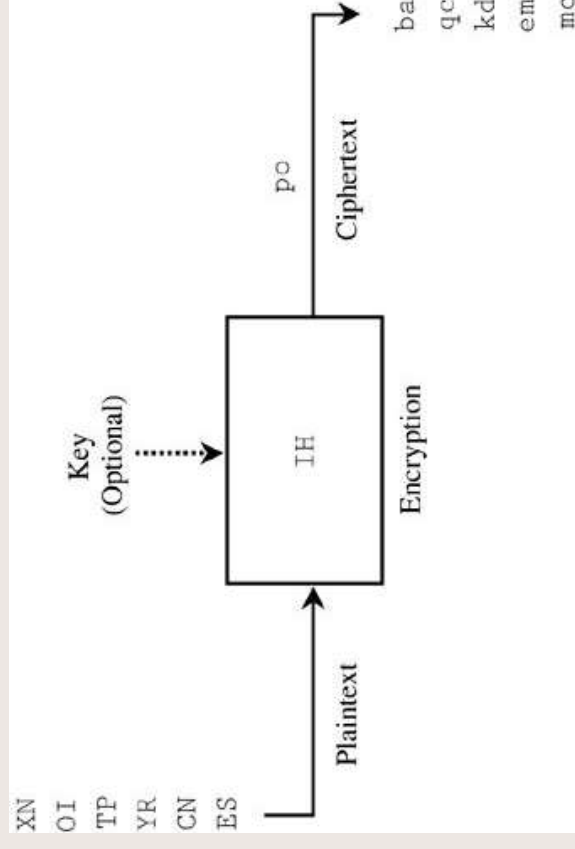
1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Z

- 2- Complete the solution of example 2 in slide number 18 by using RC4 method ?

Block Cipher

- Encrypts a group of plaintext symbols as one block.
 - It works on blocks of plaintext and produce blocks of ciphertext
 - The columnar transposition is an example of block ciphers
- A block cipher might take a 128-bit block of plaintext as input, and output.



Block Cipher Modes

How should multiple blocks be encrypted with a block cipher?

- **Electronic Codebook (ECB) mode**

creates a fixed mapping between plaintext and ciphertext blocks.

- **Cipher Block Chaining (CBC) mode**

the ciphertext from a block is used to obscure the plaintext of the next block before it is encrypted

ECB (Electronic Codebook) Mode

- Suppose we have multiple plaintext block

$P_0, P_1, P_2, \dots, P_N$, and a fixed key K

We encrypt using the formula:

$$C_i = E(P_i, K) \quad \text{for } i = 0, 1, 2, \dots, N$$

We decrypt using the formula:

$$P_i = D(C_i, K) \quad \text{for } i = 0, 1, 2, \dots, N$$

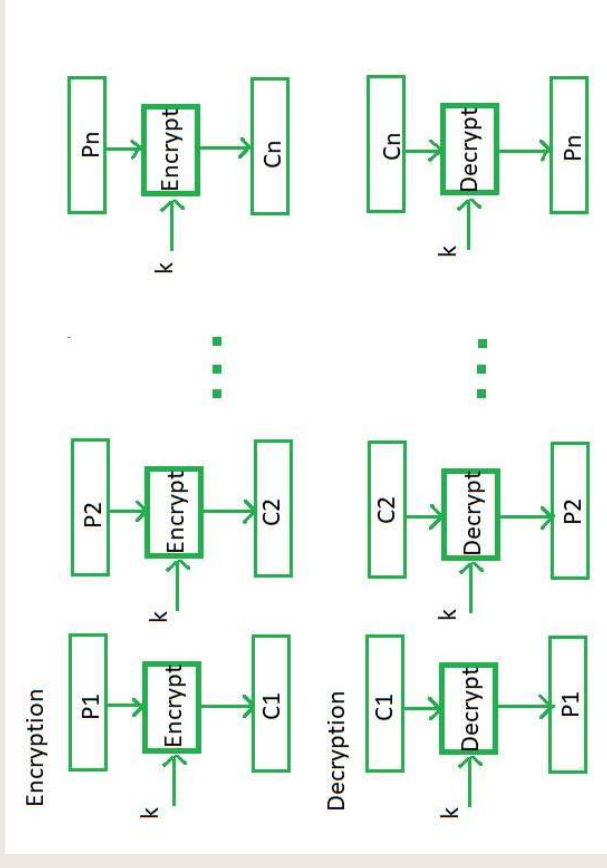
- Problem:

in case $C_i = C_j$ the attacker knows that $P_i = P_j$

Solution:

add a random text to the

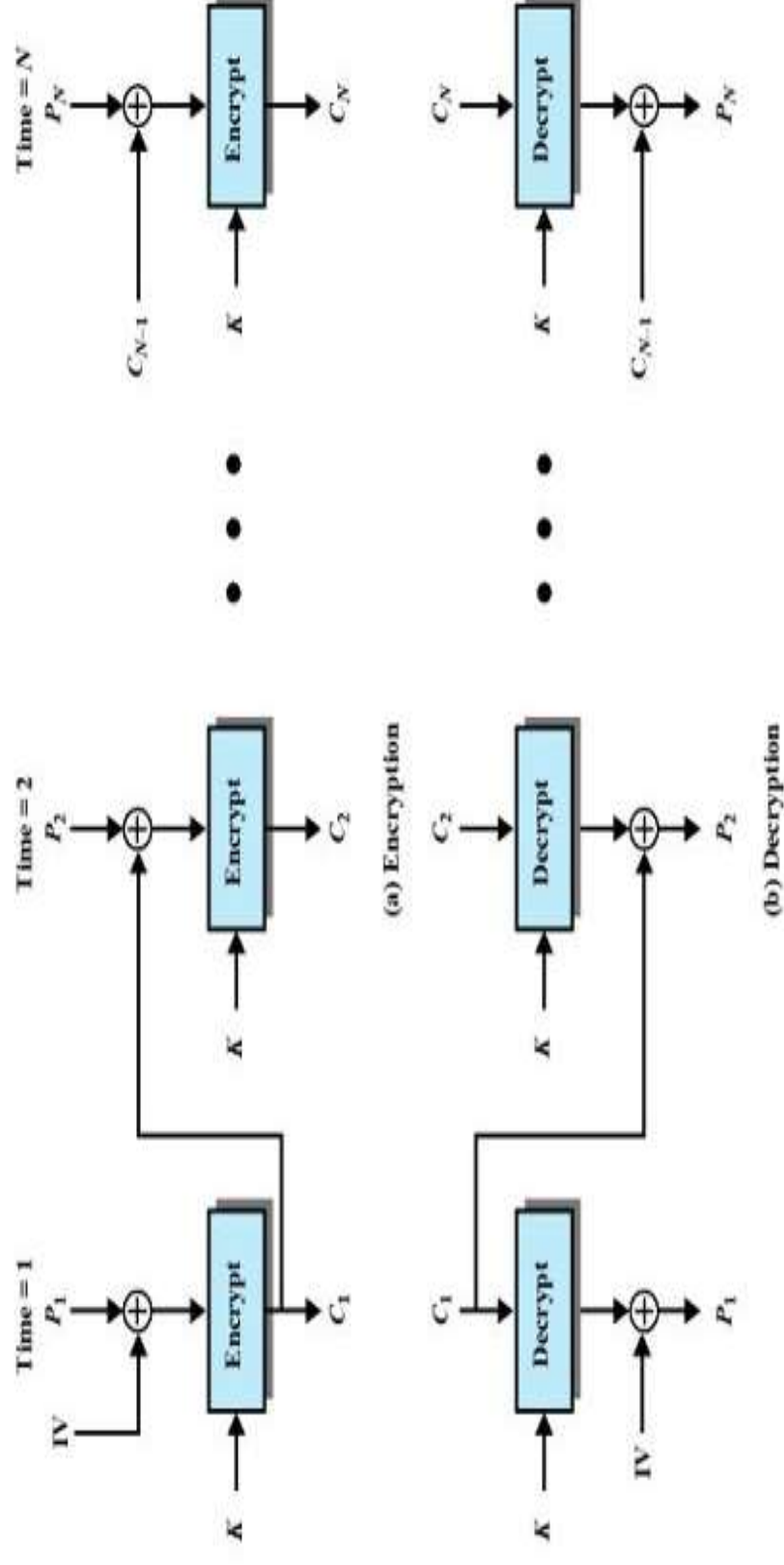
plaintexts $P_i = P_j$, so $C_i \neq C_j$



CBC (Cipher Block Chaining) Mode

- The plaintext is broken into blocks $P_1, P_2, \dots, P_N$.
- Each block of plaintext is XORed (chaining) with the previous ciphertext block before encryption
 - The formulas of encryption and decryption are:
$$C_i = E(P_i \oplus C_{i-1}, K) \text{ for } i = 0, 1, 2, \dots, N$$
$$P_i = D(C_i, K) \oplus C_{i-1} \text{ for } i = 0, 1, 2, \dots, N$$
 - What is about C_0 ?
 - An initialization vector, or IV, is used to take the place of C_{-1} .
 - IV should be randomly selected, and it sends as the first block.

CBC Mode Encryption and Decryption



CBC (Cipher Block Chaining) Mode

Problem: if a single-bit error causes two entire blocks to be garbled; it is a big problem in wireless environment.

Suppose the ciphertext block C_i is garbled to, say, $G \neq C_i$. Then

$$P_i \neq D(G, K) \oplus C_{i-1} \text{ and } P_{i+1} \neq D(C_{i+1}, K) \oplus G$$

but

$$P_{i+2} = D(C_{i+2}, K) \oplus C_{i+1}$$

and all subsequent blocks are decrypted correctly. That is, each plaintext block only depends on two consecutive ciphertext blocks, so errors do not propagate beyond two blocks.

Solution: Use stream ciphers.

Block Cipher – Advantages (Integrity)

- Block ciphers used for confidentiality and integrity.
- A message authentication code, or MAC, uses a block cipher to ensure data integrity by encrypting the data in CBC mode, and use the final block only, called MAC.
 $C_0 = E(P_0 \oplus IV, K), C_1 = E(P_1 \oplus C_0, K), \dots$
 $C_{N-1} = E(P_{N-1} \oplus C_{N-2}, K) = \text{MAC}$
- The recipient repeats the process on the plaintext and re-calculate the MAC.

Why does this MAC computation work? Suppose Alice sends

$IV, P_0, P_1, P_2, P_3, \text{MAC}$

to Bob. Now, if Trudy changes plaintext block P_1 to, say, Q during transmission, then when Bob attempts to verify the MAC, he computes

$$C_0 = E(P_0 \oplus IV, K), \quad \tilde{C}_1 = E(Q \oplus C_0, K), \quad \tilde{C}_2 = E(P_2 \oplus \tilde{C}_1, K), \\ \tilde{C}_3 = E(P_3 \oplus \tilde{C}_2, K) = \text{"MAC"} \neq \text{MAC}.$$

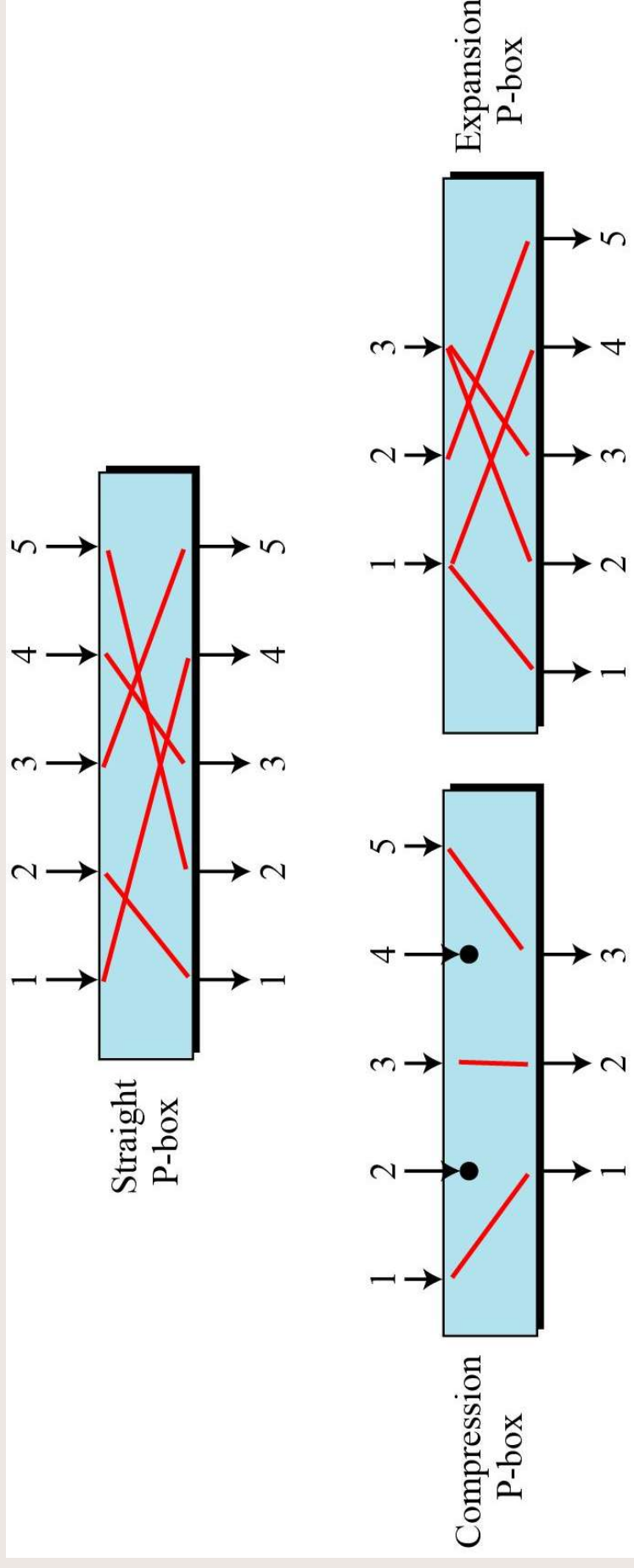
The reason this works is because any change to a plaintext block propagates into subsequent blocks in the process of computing the MAC.

Components of a Modern Block Cipher

- ☐ *P-Boxes*
- ☐ *S-Boxes*
- ☐ *Exclusive OR*
- ☐ *Circular shift*
- ☐ *Swap*
- ☐ *Split and Combine*

P-Box

- A P-box (permutation box) parallels the traditional transposition cipher for characters. It transposes bits.



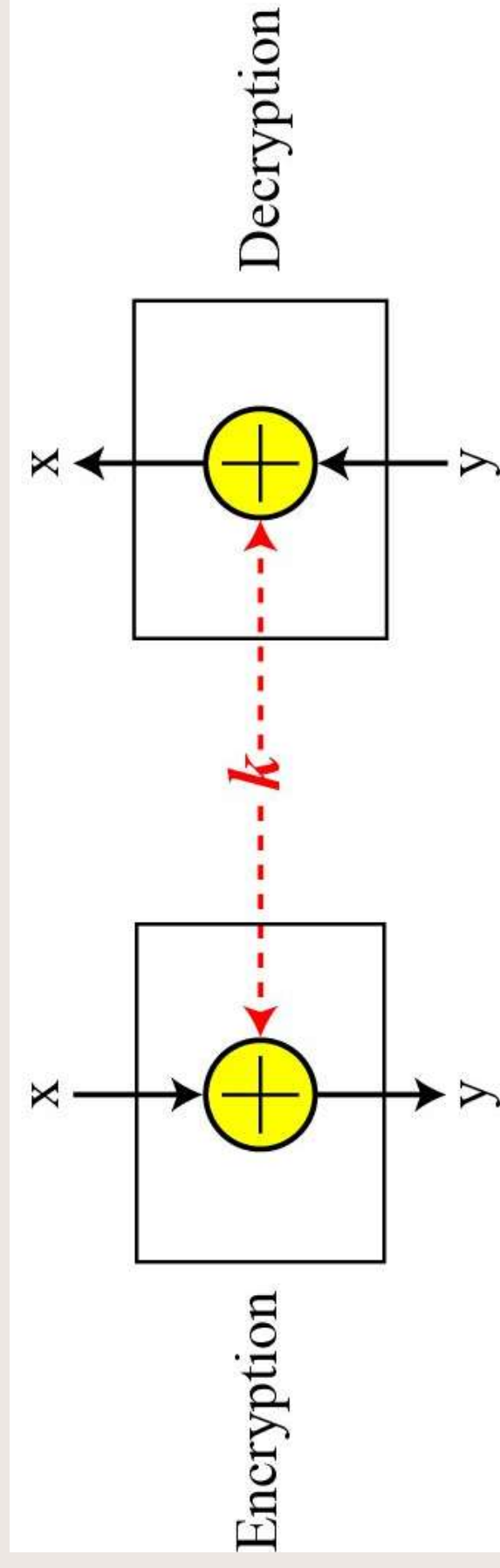
Permutation Table for a 64 Bit Straight P-Box

58	50	42	34	26	18	10	02	60	52	44	36	28	20	12	04
62	54	46	38	30	22	14	06	64	56	48	40	32	24	16	08
57	49	41	33	25	17	09	01	59	51	43	35	27	19	11	03
61	53	45	37	29	21	13	05	63	55	47	39	31	23	15	07

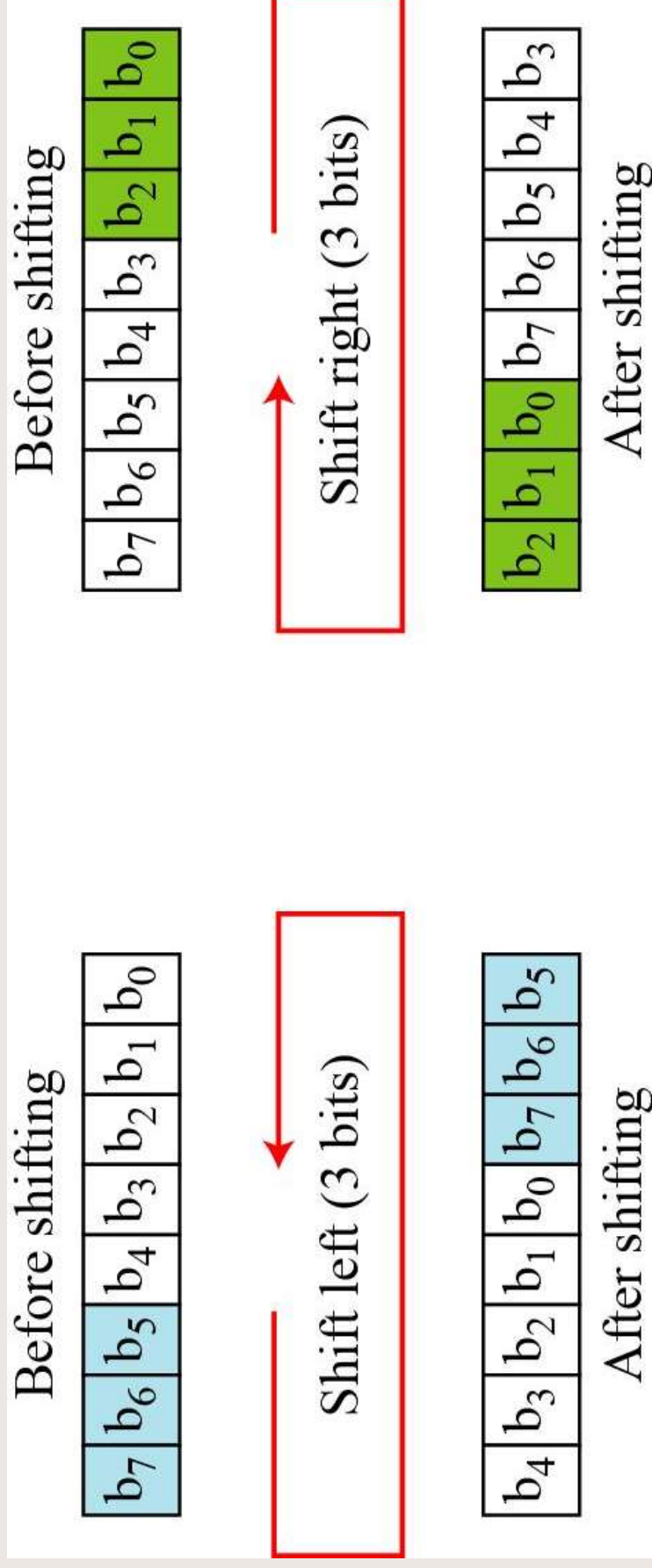
S-Box

- *An S-box (substitution box) can be thought of as a miniature substitution cipher.*
- *Like P-Box, S-Box can be straight, compression or expansion*

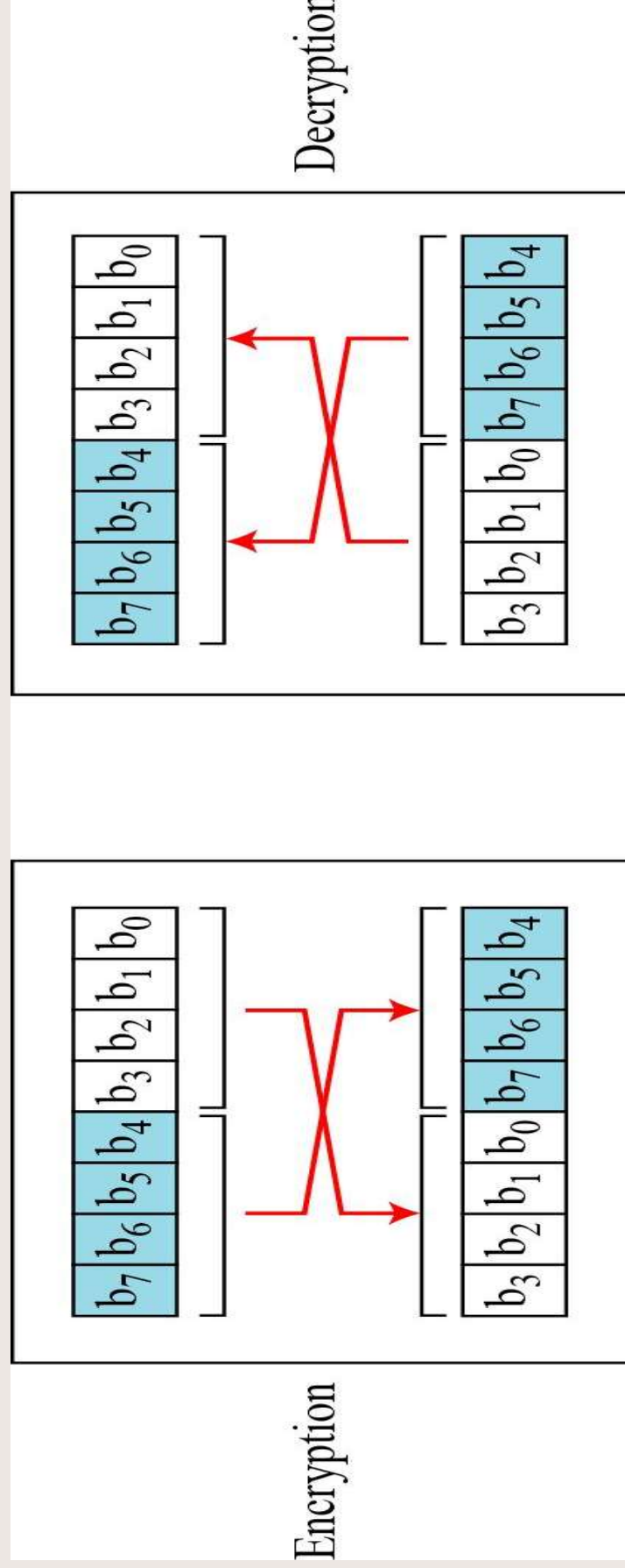
XOR



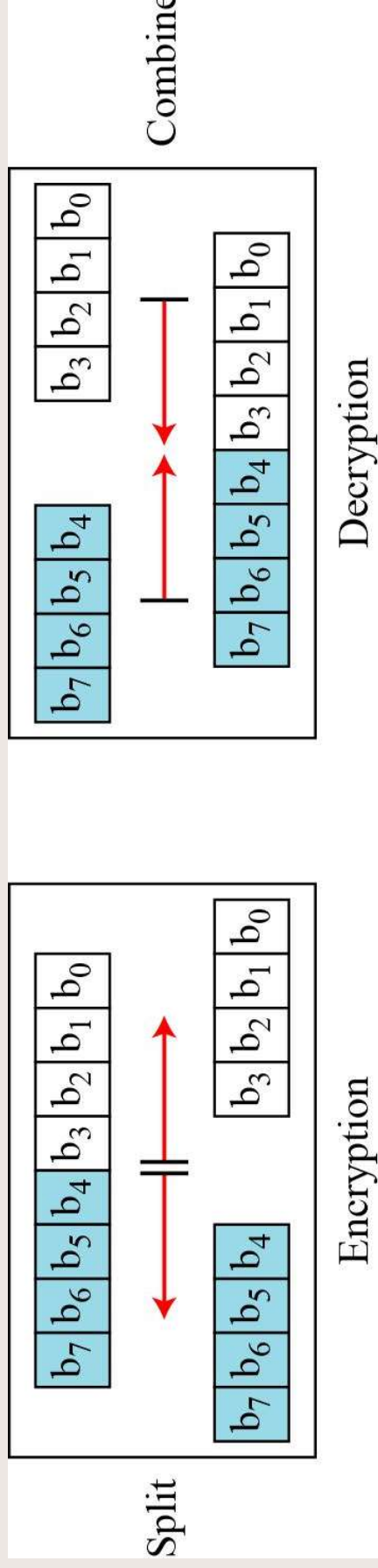
Circular Shift



Swap



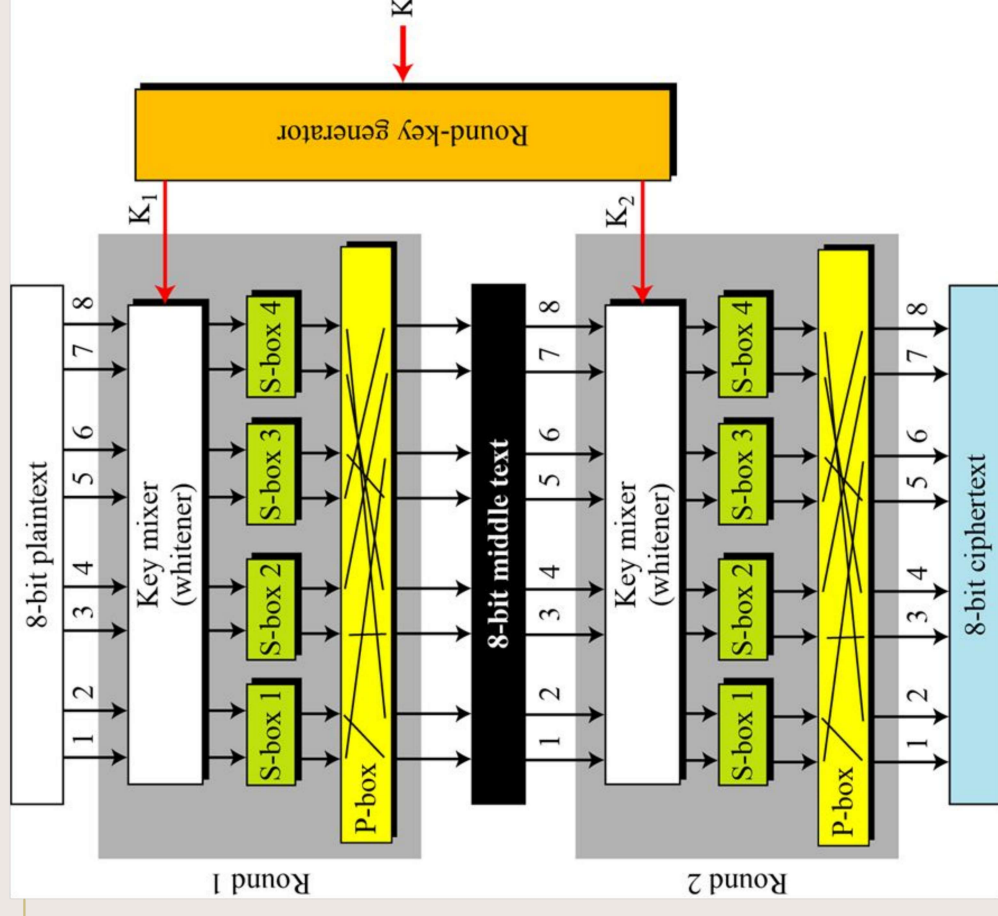
Split and Combine



Product Cipher

- Shannon introduced the concept of a product cipher. A product cipher is a complex cipher combining substitution, permutation, and other components.

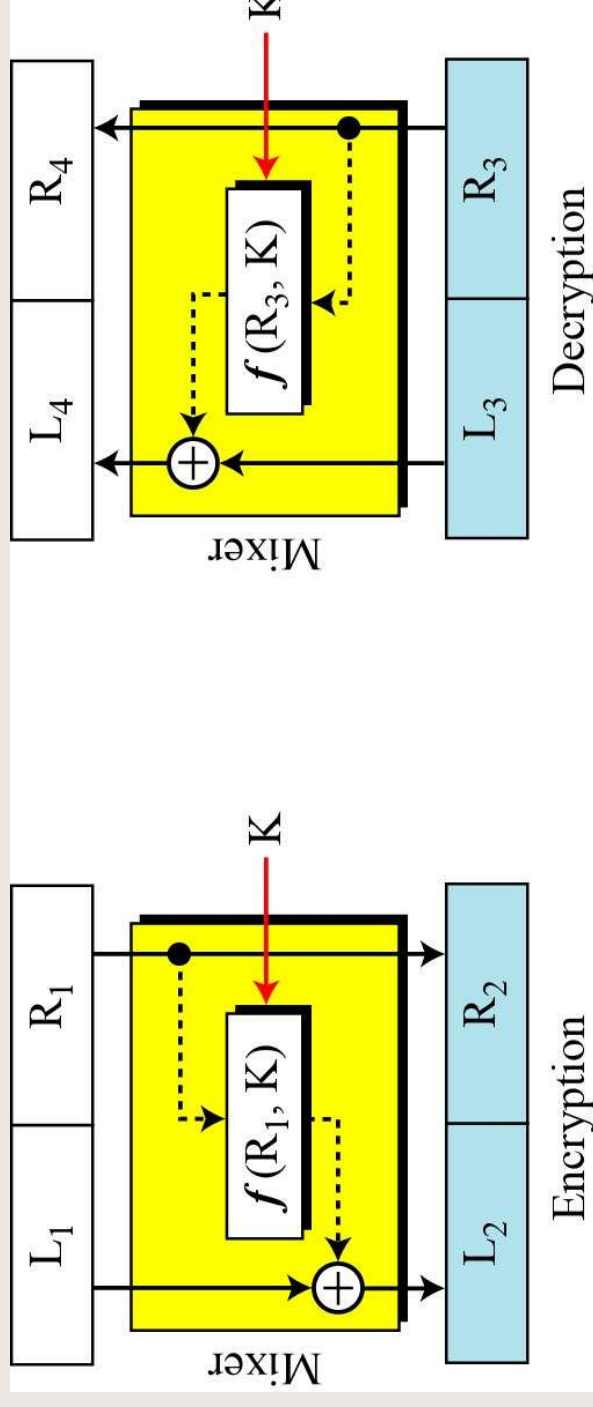
Product Cipher



Fiestel Cipher

39

- Partitions input block into two halves that are processed through multiple rounds which perform a substitution on left data half based on round function of right half & subkey then have permutation swapping halves
- $F(R_{i-1}, K_i) = P\text{-box}(S\text{-boxes}(\text{Expand}(R_{i-1}) \oplus K_i))$



Fiestel Cipher

- **Encryption:** Partitions plaintext block into left and right halves $P = (L_0, R_0)$ that are processed through multiple rounds which perform a substitution on left data half based on round function of right half & subkey (K_i) then have permutation swapping halves as

$$\left. \begin{array}{l} L_i = R_{i-1} \\ R_i = L_{i-1} \oplus F(R_{i-1}, K_i) \end{array} \right\} i = 1, 2, \dots, n$$

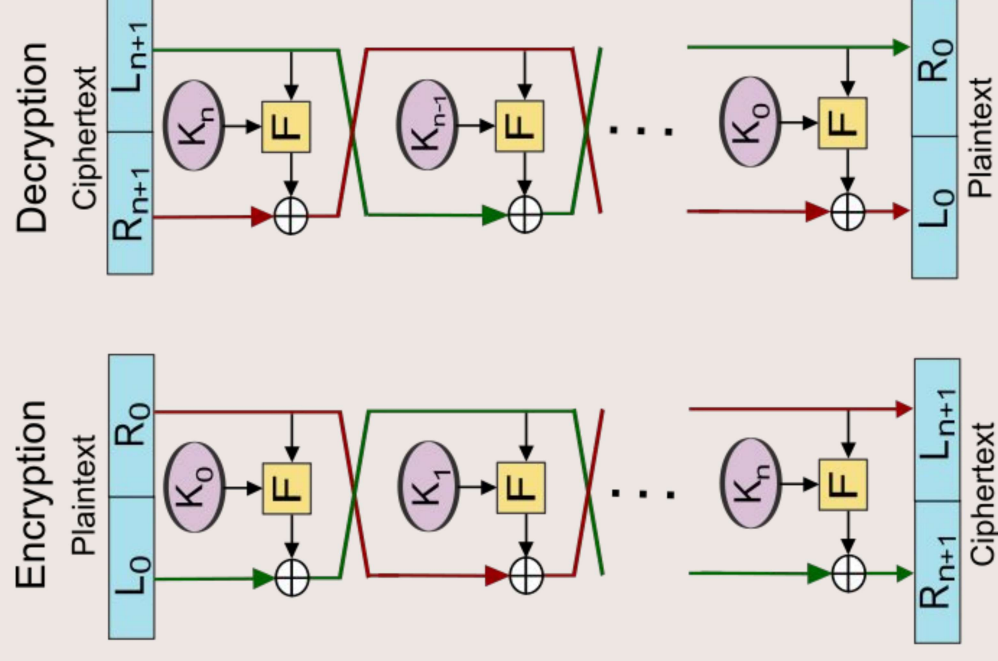
Where F is the round function

$$F(R_{i-1}, K_i) = \text{P-boxes}(\text{S-boxes}(\text{Exapnd}(R_{i-1}) \oplus K_i))$$

- **Decryption:**

$$\left. \begin{array}{l} R_i = L_{i-1} \\ L_i = R_{i-1} \oplus F(R_{i-1}, K_i) \end{array} \right\} i = n, n-1, \dots, 1$$

e



Fiestel Cipher

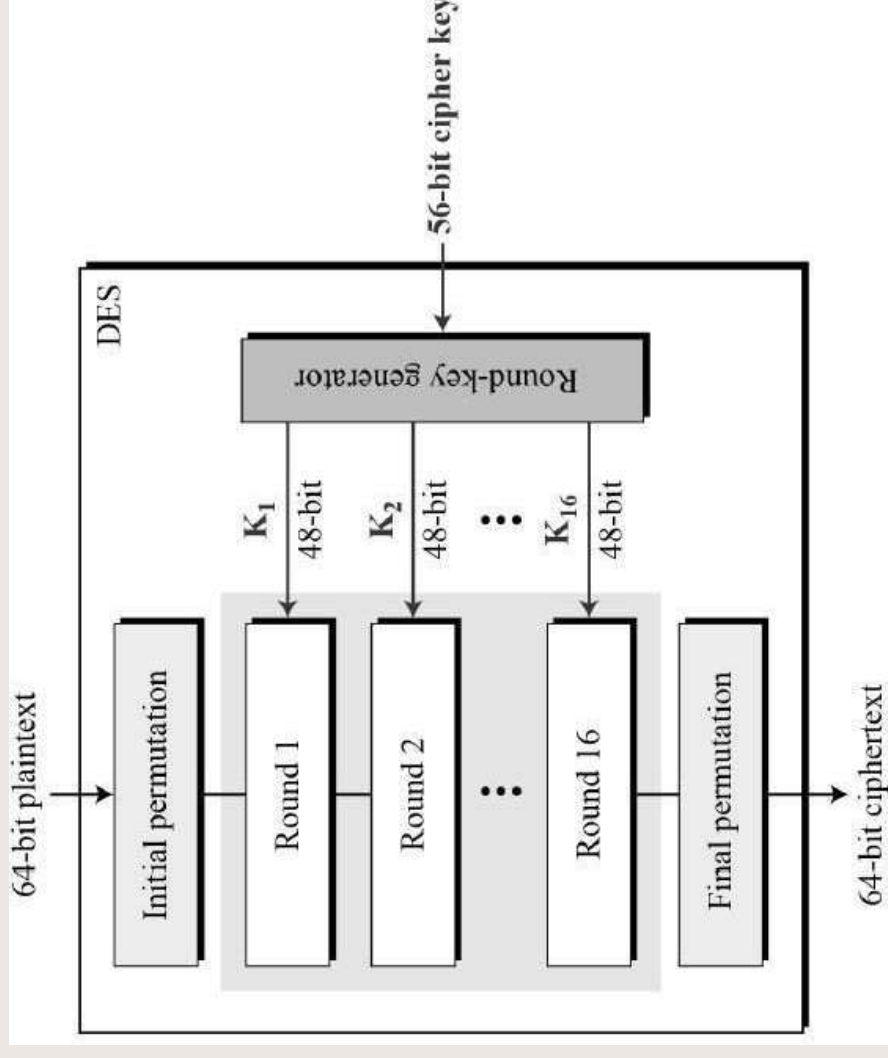
- Feistel is a general cipher design principle
- F will work in a Feistel cipher, provided that the output of F produces the correct number of bits.
- Feistel cipher will not be secure for all possible choices of F.
- For example, the round function $F(R_{i-1}, K_i) = 0$ for all R_{i-1} and K_i is a legitimate round function since we can encrypt and decrypt with this F.

Data Encryption Standard (DES)

- most widely used block cipher in world
- adopted in 1977 by National Bureau of Standards (NBS) (now NIST)
- encrypts 64-bit data using 56-bit key
- had widespread use
- DES is a Feistel cipher with the following numerology:
 - 16 rounds
 - 64-bit block length
 - 56-bit key (in reality, 64 bits, but 8 are used as parity-check bits for error control)
 - 48-bit subkeys
- has been considerable controversy over its security

DES: Rounds

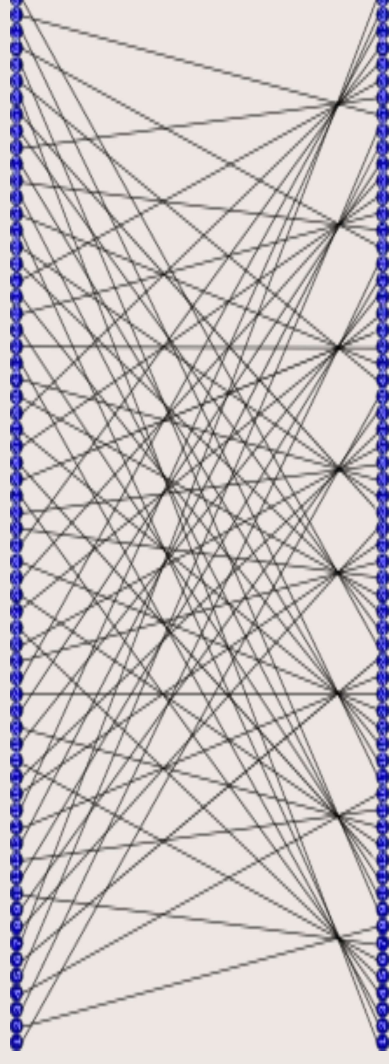
- Runs 16 rounds for each block of the plaintext
 - Each block consists of 64 bits. It uses a 56-bit key.
- DES performs an initial permutation on each block and final permutation after 16 rounds
- These permutations are inverse of each other and operate on 64 bits



Initial Permutation (IP)

- An initial Permutation (IP) function takes 64 bits block of plaintext and rearrange the bits as shown on fig.
- The IP produces two halves of the permuted block

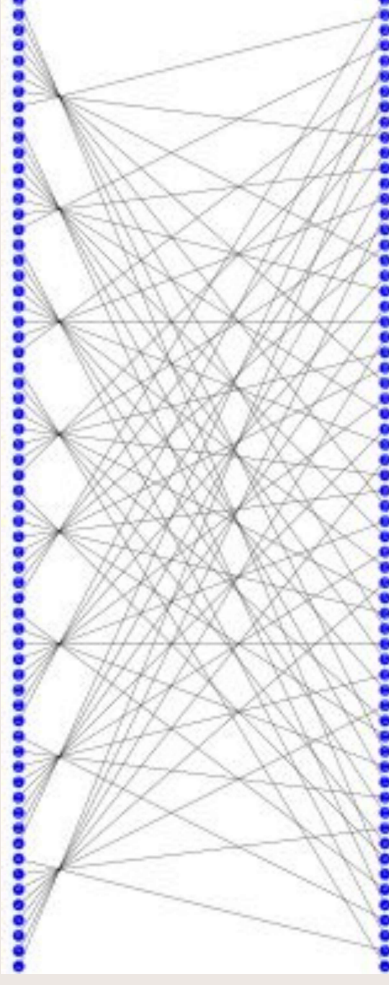
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7



Final Permutation (IP^{-1})

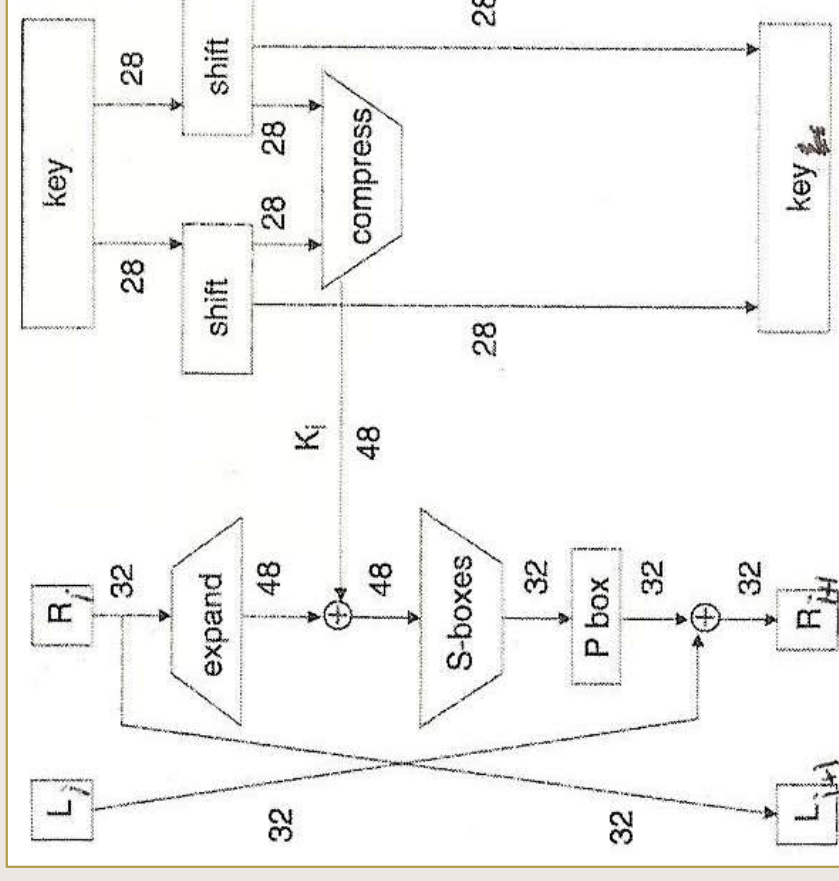
- The final permutation is the inverse of the initial permutation; the table is interpreted similarly.
- That is, the output of the Final Permutation has bit 40 of the pre output block as its first bit, bit 8 as its second bit, and so on, until bit 25 of the pre output block is the last bit of the output.

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25



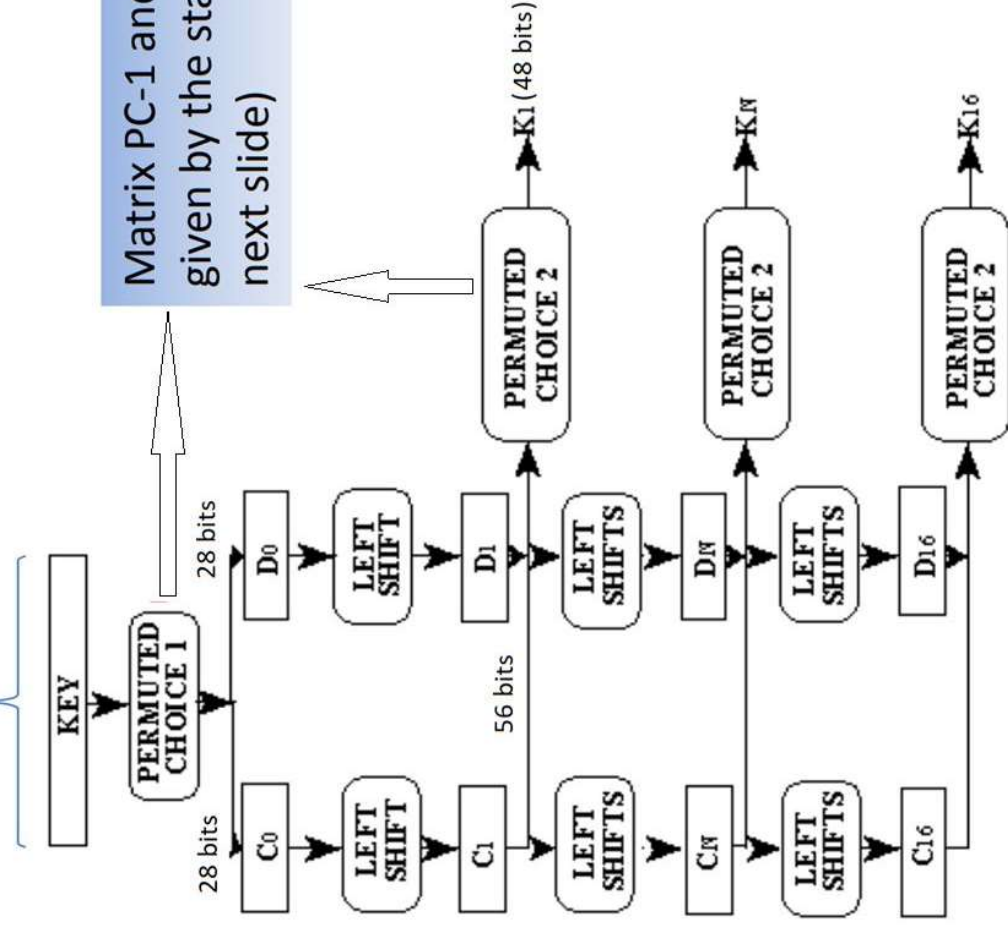
DES: One Round

- In each round of DES:
 - A. Key Generation:
 1. Dividing them into 2 halves (28bit each)
 2. Bit-shift left
 3. Compress the key (56bit $\rightarrow$ 48bit)
 - B. Block of the Plaintext:
 1. Dividing them into L_i and R_i (32bit each)
 2. Expansion permutation (32bit $\rightarrow$ 48bit)
 3. Addition of subkey (48bit $\oplus$ 48bit)
 4. S-boxes (map 6bits $\rightarrow$ 4bits)
 5. P-box (change permutation)



A: DES: Key Generation ($K_1 - K_{16}$)

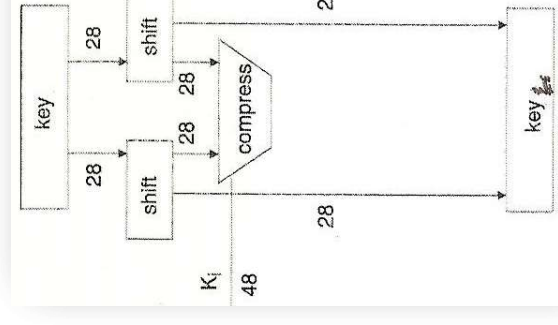
64 bit key (including parity-check bits)



Matrix PC-1 and PC-2 are given by the standard (see next slide)

$$\begin{aligned} C_i &= LS_i(C_{i-1}) \\ D_i &= LS_i(D_{i-1}) \\ K_i &= PC-2(C_i D_i) \end{aligned}$$

LS=Left Shift
-shift *one* position
if $i=1, 2, 9$ or 16
-shift *two* positions
otherwise



DES: Key Schedule

Key:

1. Divide the key into 2 parts according to *Parity-bit drop table* (namely, bits 8,16, 4,32,40,48,56,64 are dropped), *not appear in Permuted Choice: PC-1*:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64

Figure - discarding of every 8th bit of original key



2. Bit-shift left (circular) according to the round number:

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bit shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

3. PC-2 selects the 48-bit subkey for each round from the 56-bit key-schedule state



Left															
57	49	41	33	25	17	9									
1	58	50	42	34	26	18									
10	2	59	51	43	35	27									
19	11	3	60	52	44	36									
Right															
63	55	47	39	31	23	15									
7	62	54	46	38	30	22									
14	6	61	53	45	37	29									
21	13	5	28	20	12	4									

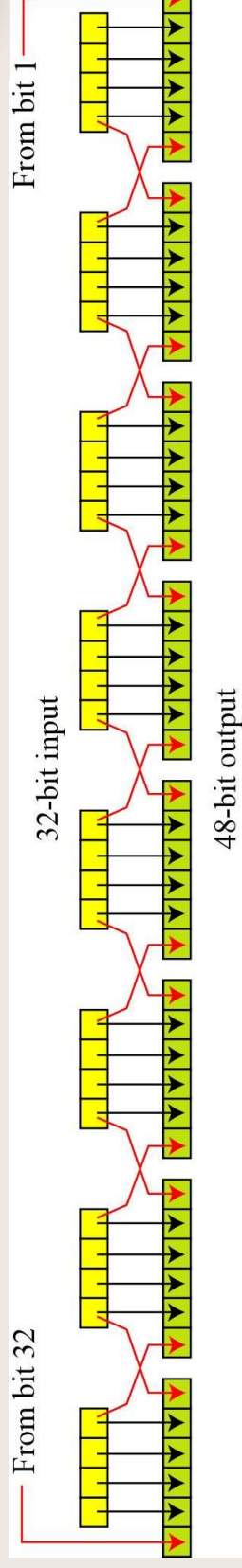
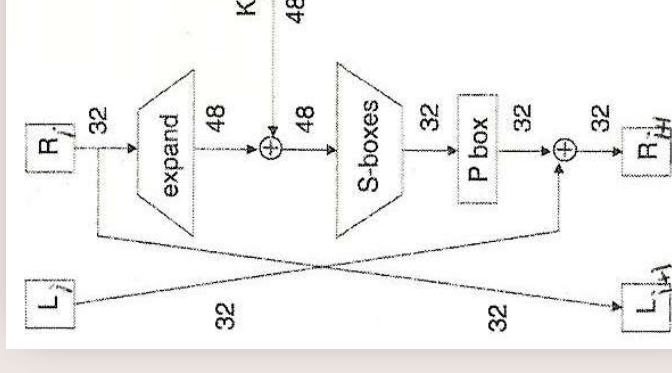
14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

B: DES: Expansion

B. Plaintext

1. Expansion function which takes a block of 32 bits as input and produces a block of 48 bits as output

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

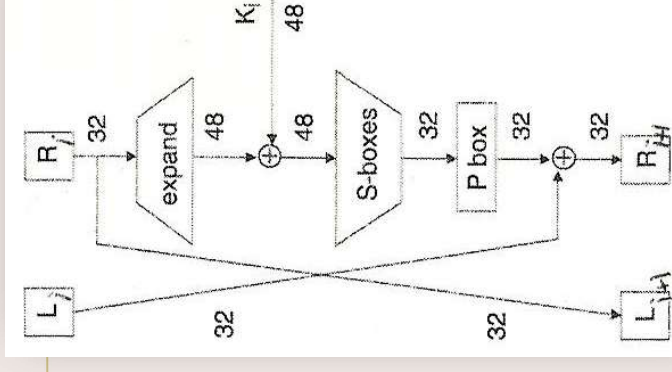


DES:XOR

2. Addition of Subkey:

For each round i , we have the right part of the plaintext R_i after the expansion, and the subkey for this round K_i (both of them are 48 bits.)

Find: $K_i \oplus R_i$



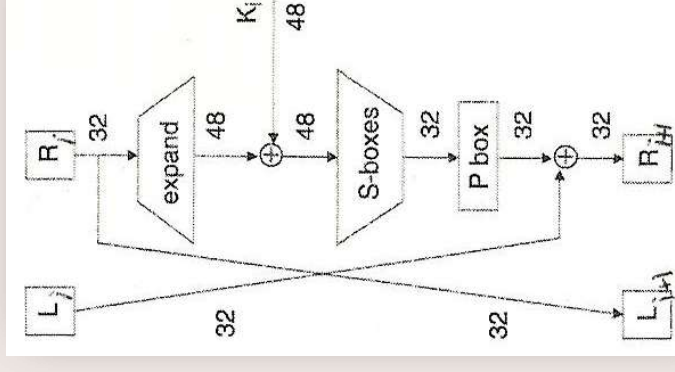
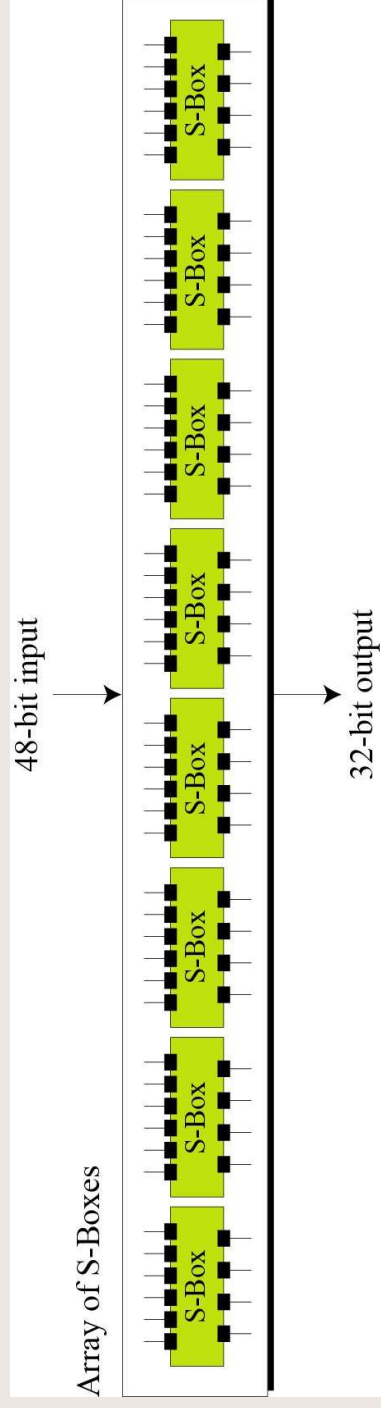
DES: S-boxes

3. S-boxes:

The main purpose of S-boxes is to offer protection against cryptanalytic. There are 8 distinct S-boxes that are used in each round.

In: 6 bits $\times$ 8 S-box = 48 bits

Out: 4 bits $\times$ 8 S-box = 32 bits



One S-Box Representation



- S = matrix 4×16 , values from 0 to 15
- B (6 bit long) = $b_1 b_2 b_3 b_4 b_5 b_6$
 - $b_1 b_6 \rightarrow r$ = row of the matrix (2 bits: 0,1,2,3)
 - $b_2 b_3 b_4 b_5 \rightarrow c$ = column of the matrix (4 bits: 0,1,...,15)
- C (4 bit long) = Binary representation of $S(r, c)$

Example 1 (S.Box): DES

?The input to S-box 1 is 100011. What is the output

If we write the first and the sixth bits together, we get 11 in binary, which is 3 in decimal. The remaining bits are 0001 in binary, which is 1 in decimal. We look for the value in row 3, column 1, in Table 3.1 (S-box 1). The result is 12 in decimal, which in binary is 1100. So the input 100011 yields the output 1100.

Table 3.1 S-box 1

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

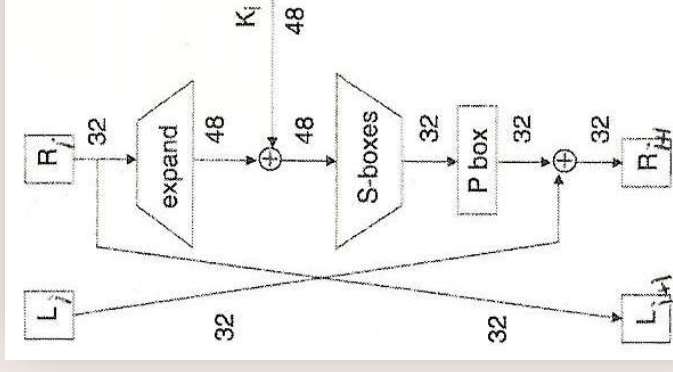
DES: P-boxes

54

4. P-boxes:

For each round i , there are 32 bits out from the S-Boxes. Change their permutation to:

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25



Example1 (P.Box): DES

55

The following permutation table is used to permute an 8-bit input:

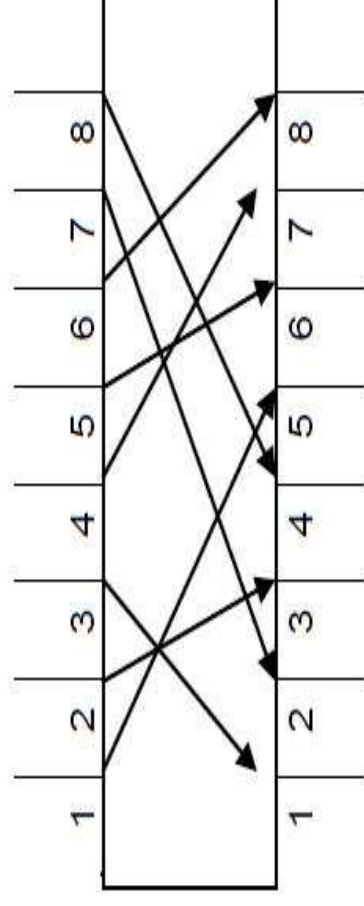
IP = [3 7 2 8 1 5 4 6]

- A. Draw the internal connections of both the corresponding IP-box and the inverse IP-box (IP-1 -box).
- B. Find the inverse permutation table IP-1.

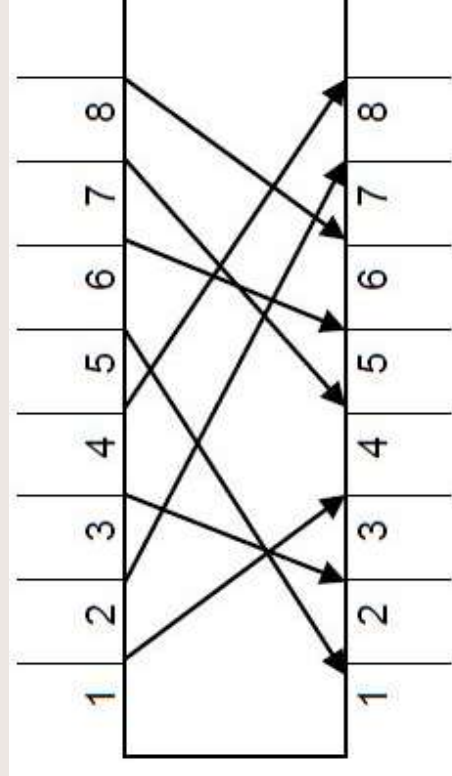
Example (1): DES

56

IP – box?



Inverse IP – box?



Inverse IP Permutation = [5 3 1 7 6 8 2 4]

C.1. Symmetric Key Algorithms

2. Block Cipher – Example (1): DES

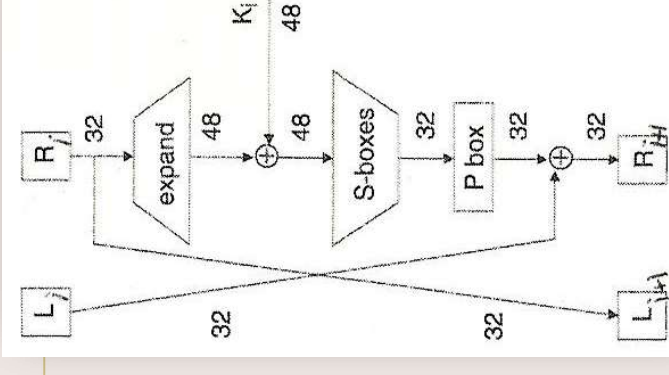
57

5. Constructing the ciphertext :

For each round i

$$L(i) = R(i-1)$$

$$R(i) = L(i-1) \oplus P(S(E(R(i-1)) \oplus K(i)))$$



Example 2: DES

- Plain text = p= [000000000]
- P4= [2431]
- IP= [26314857]; IP-1=[41357286]
- E/P= [41232341]
- P8=[637485109]

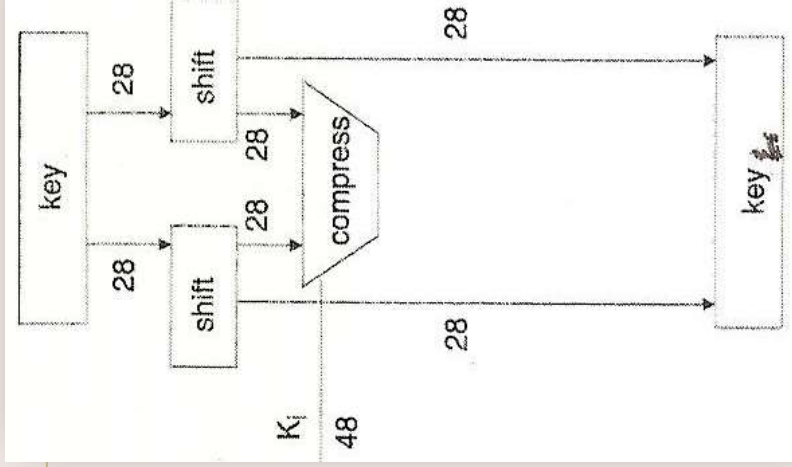
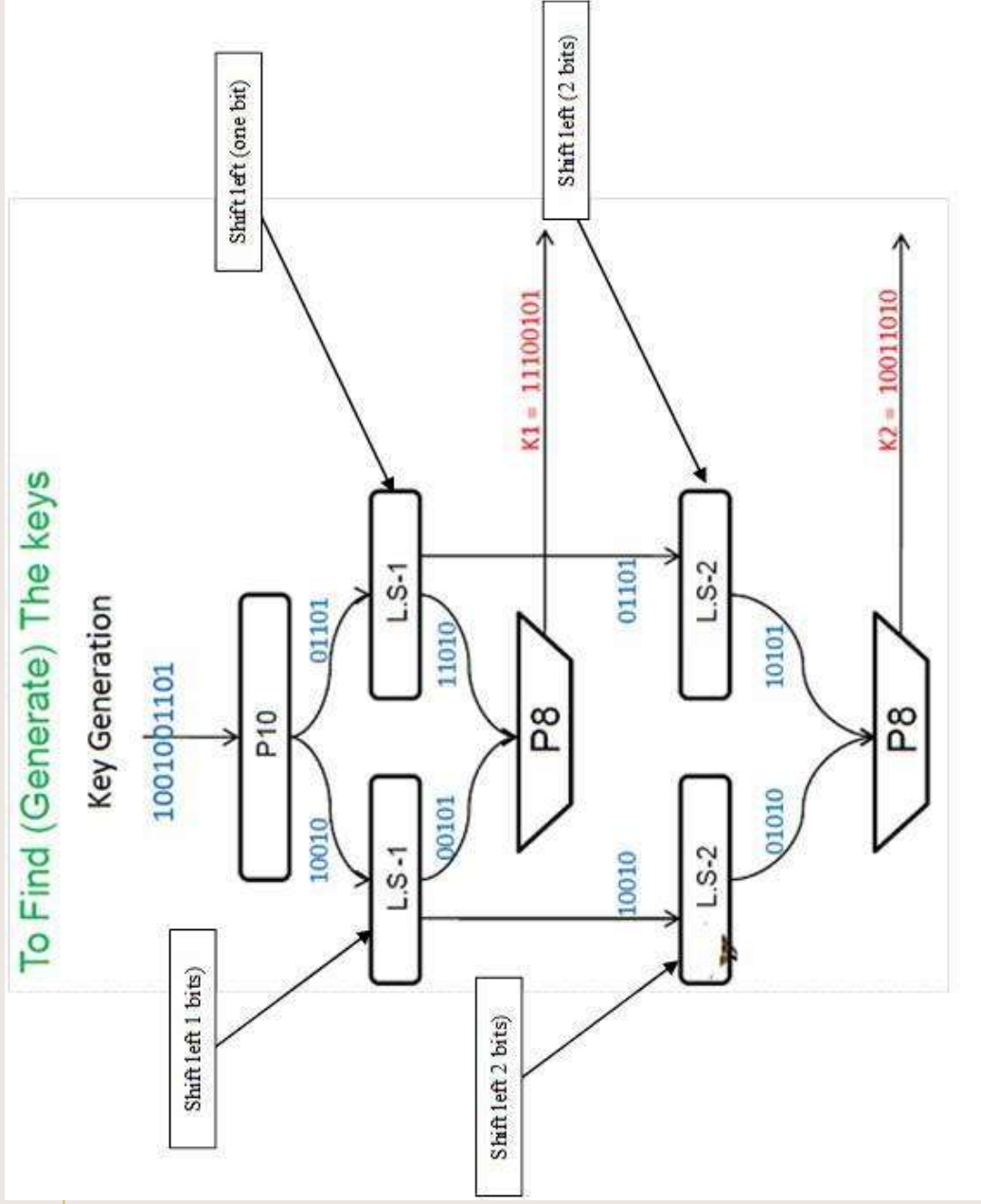
S0=

	00	01	10	11
00	1	0	3	2
01	3	2	1	0
10	0	2	1	3
11	3	1	3	2

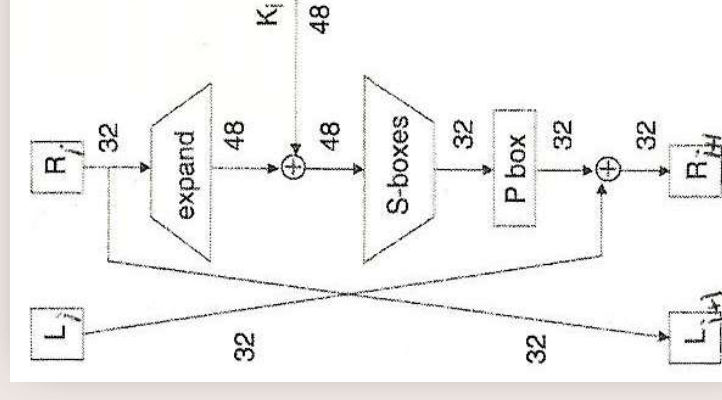
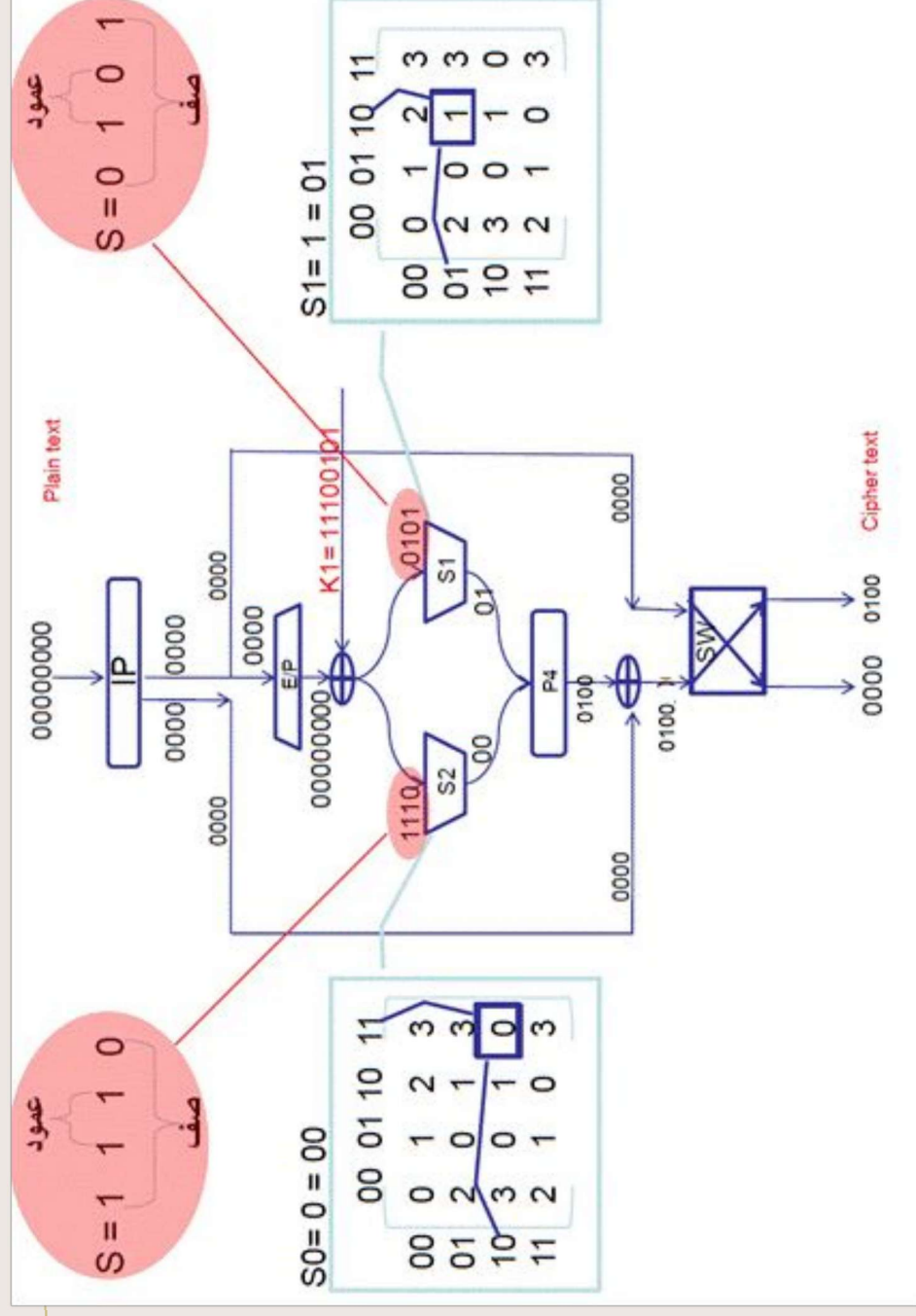
S1=

	00	01	10	11
00	0	1	2	3
01	2	0	1	3
10	3	0	1	0
11	2	1	0	3

Solution: Example 2 (DES)



Solution: Example 2 (DES)



DES Weak Keys

- What are DES Weak Keys? How they influence DES working?
- DES has 4 weak keys (64-bit)
 - 01010101 01010101
 - FEF EFEFE FEF EFEFE
 - E0E0E0E0 F1F1F1F1
 - 1F1F1F1F 0E0E0E0E
- Using weak keys, the outcome of the Permuted Choice 1 (PC1) in the DES key schedule leads to round keys (K_1 --- K_{16}) being either all zeros, all ones or alternating zero-one patterns
- Since all the subkeys are identical, and DES is a Feistel network, the encryption function becomes self-inverting; that is, encrypting twice with a weak key K produces the original plaintext. – $E_K(E_K(x))=x$ for all x , i.e., the encryption and the decryption are the same

Triple DES

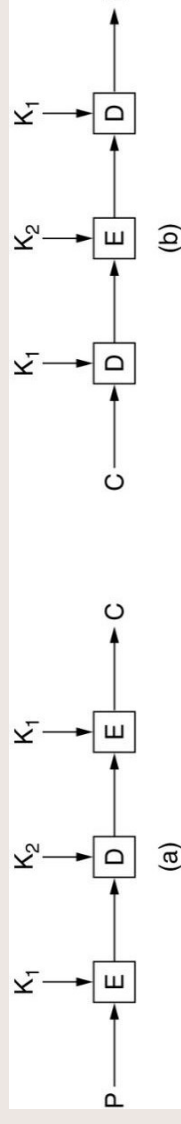
- Let P be a block of plaintext, K a key, and C the corresponding block of ciphertext. For DES, C and P are each 64 bits, while K is 56 bits –
- Triple DES effectively increases the key length. It uses two keys and three

stages:

$C = E(P, K)$; encrypt the plaintext P with the key K

$P = D(C, K)$; for decryption

- It takes 2^{55} key search to attack DES
- Moving to encryption with two keys according to:
 1. $C = E(E(P, K_1), K_2)$ is not secure, WHY? require 2^{55} to break
 2. $C = E(D(E(P, K_1), K_2), K_1)$



Advanced Encryption Standard (AES)

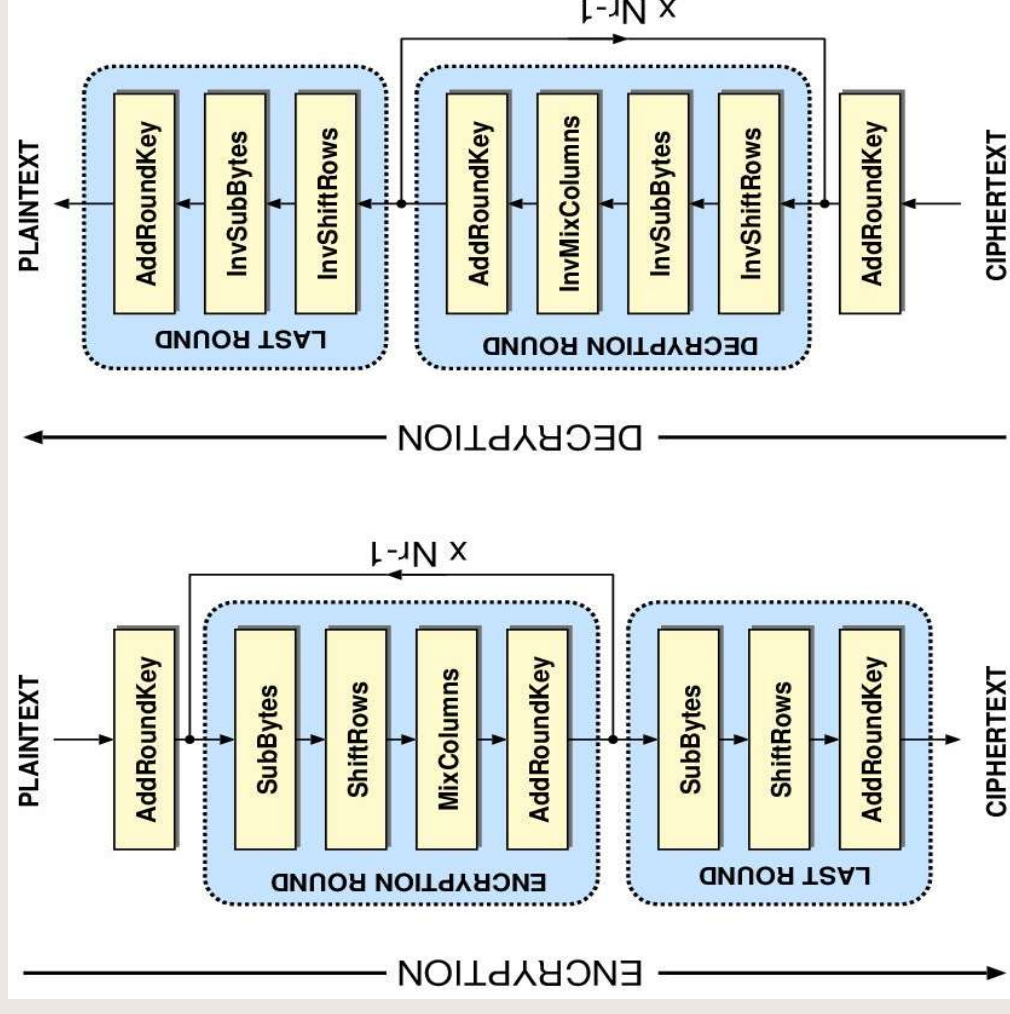
- A call for crypto proposals by NIST in the early 1990s.
- 22 submissions, 7 did not satisfy all requirements, 15 submissions accepted.
- 5 finalists was announced in 1999: Mars, RC6, Rijndael, Serpent, Twofish.
Winner: Rijndael.
- In Oct. 2000, Based on a competition won by Rijmen and Daemen (Rijndael) from Belgium as the AES
- The AES is an iterated block cipher
- Unlike DES, the AES algorithm is not a Feistel cipher.

Applications of AES

- AES is widely used in many applications which require secure data storage and transmission. Some common use cases include:
- **Wireless security:** AES is used in securing wireless networks, such as Wi-Fi networks, to ensure data confidentiality and prevent unauthorized access.
- **Database Encryption:** AES can be applied to encrypt sensitive data stored in databases. This helps protect personal information, financial records, and other confidential data from unauthorized access in case of a data breach.
- **Secure communications:** AES is widely used in protocols like such as internet communications, email, instant messaging, and voice/video calls. It ensures that the data remains confidential.
- **Data storage:** AES is used to encrypt sensitive data stored on hard drives, USB drives, and other storage media, protecting it from unauthorized access in case of loss or theft.
- **Virtual Private Networks (VPNs):** AES is commonly used in VPN protocols to secure the communication between a user's device and a remote server. It ensures that data sent and received through the VPN remains private and cannot be deciphered by eavesdroppers.
- **Secure Storage of Passwords:** AES encryption is commonly employed to store passwords securely. Instead of storing plaintext passwords, the encrypted version is stored. This adds an extra layer of security and protects user credentials in case of unauthorized access to the storage.
- **File and Disk Encryption:** AES is used to encrypt files and folders on computers, external storage devices, and cloud storage. It protects sensitive data stored on devices or during data transfer to prevent unauthorized access.

AES: SALIENT FEATURES

- The block size is 128 bits
- Key sizes: 128, 192, 256 (AES-128, AES-192, AES-256)
- AES runs for 10, 12, and 14 rounds using 128, 192 and 256 bits keys respectively
- Except for the last round in each case, all other rounds are identical.
- # Rounds $N_r = 6 + \max\{N_b, N_k\}$
 - $N_b = 32$ -bit words in the block
 - $N_k = 32$ -bit words in key



AES: SALIENT FEATURES

Each round consists of four functions, in three layers

- **ByteSub (nonlinear layer)**: one single-byte based substitution step,
- **ShiftRow (linear mixing layer)**: a row-wise permutation step
- **MixColumn (nonlinear layer)**: a column-wise mixing step,
- **AddRoundKey (key addition layer)**: the addition of the round key

AES Data Structure: Input State Array

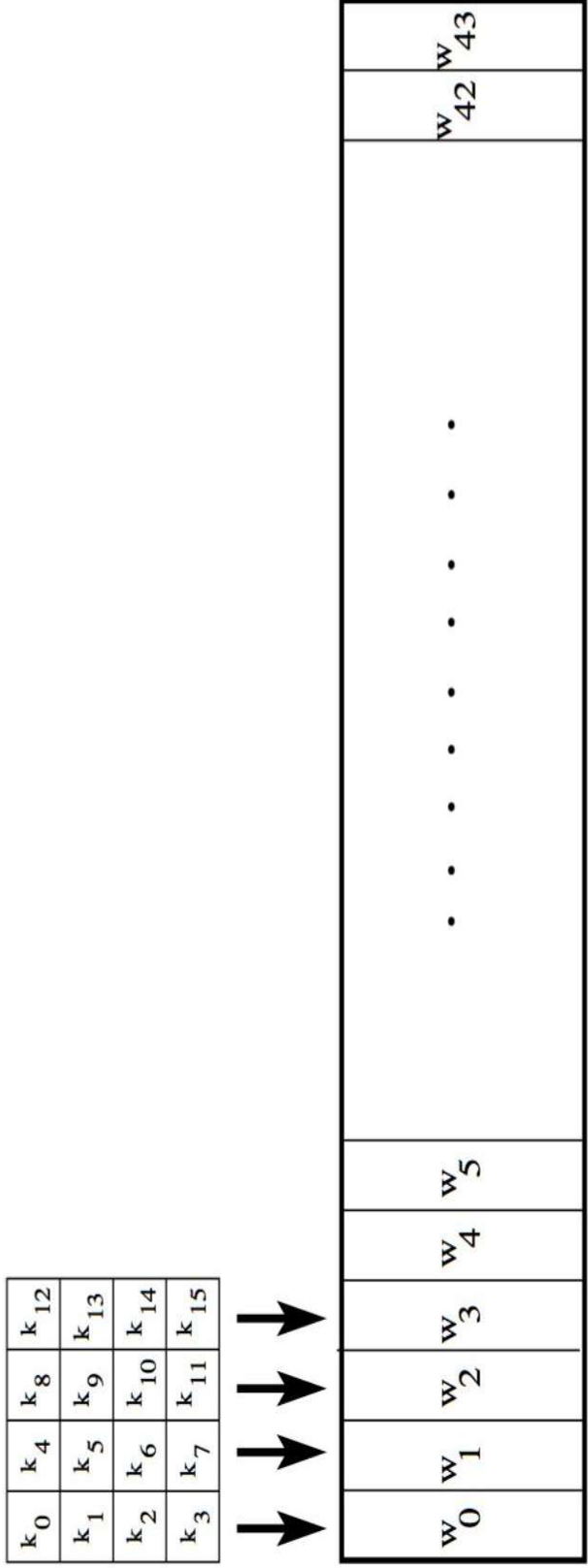
- At the beginning, a 128-bit block is arranged in form of a 4×4 array of bytes as follows

$$\begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix}$$

- Notice that the first four bytes of a 128-bit input block occupy the first column in the 4×4 array of bytes. The next four bytes occupy the second column, and so on.
- The 4×4 array of bytes shown above is referred to as the state array, which is modified at each stage of encryption.
- A word consists of four bytes, that is 32 bits. Therefore, each column of the state array is a word, as is each row.

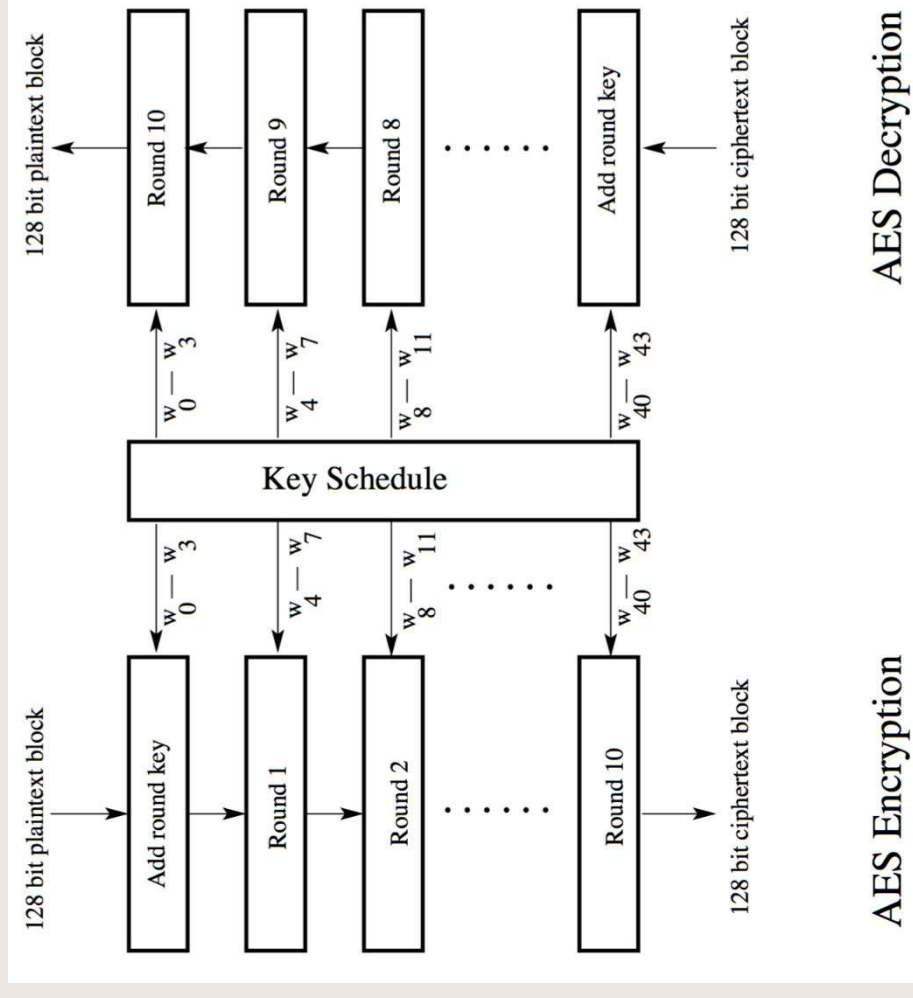
AES Data Structure: The Keys and Expanded Keys

- Assuming a 128-bit key, the key is also arranged in the form of an array of 4×4 bytes.
- The four column words of the key array are expanded into a schedule of 44 words. (**As to how exactly this is done, explain that later**)
-



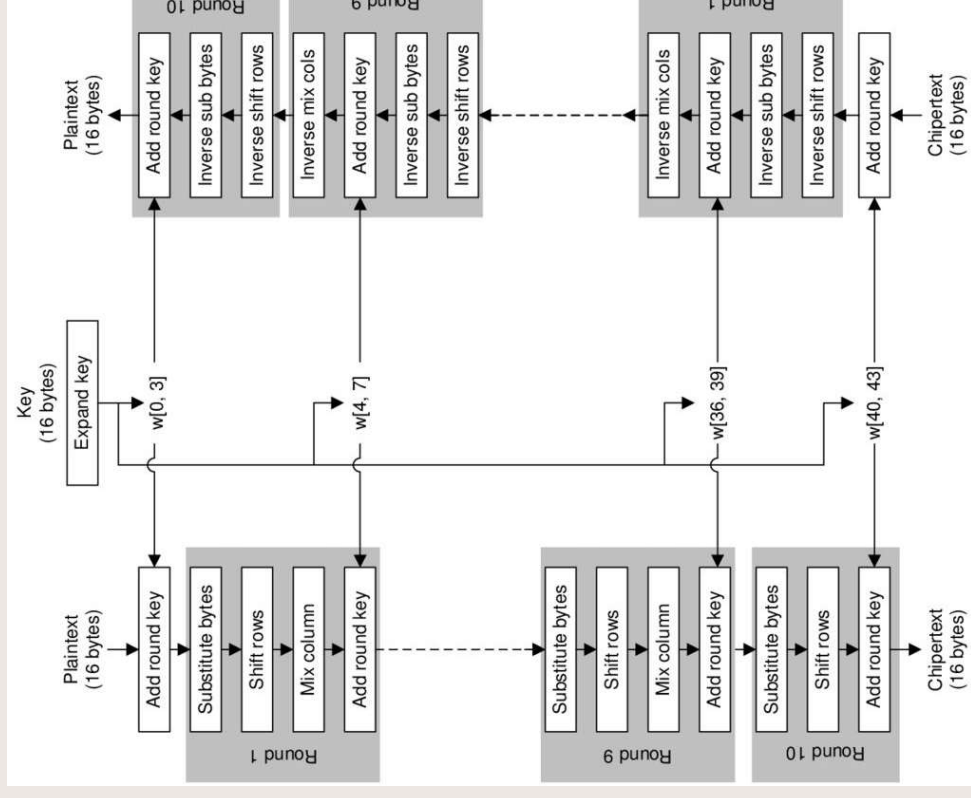
AES Data Structure: The Keys and Expanded Keys

- Each round consumes four words from the key schedule.
- The first four words are used for XORing to the input state array before any round-based processing can begin, and the remaining 40 words used for the ten rounds of processing that are required for the case a 128-bit encryption key.



AES Encryption

- A. Before any round-based processing for encryption can begin, the input state array is XORed with the first four words of the key schedule.
- B. For encryption, each round consists of the following four steps:
- 1) Substitute bytes,
 - 2) Shift rows,
 - 3) Mix columns, and
 - 4) Add round key.
- C. The last step consists of XORing the output of the previous three steps with four words from the key schedule. The last round for encryption does not involve the “Mix columns” step.



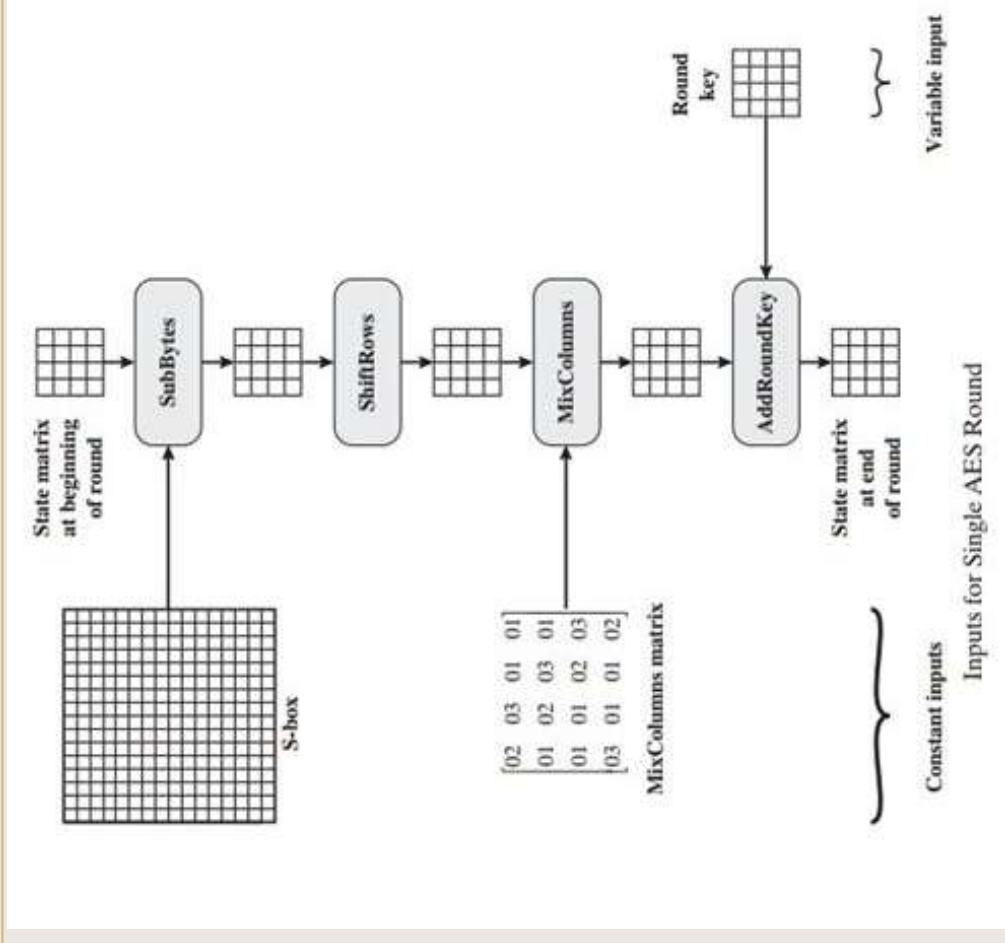
A. First Add Round Key Step

- At the beginning, the input state array is XORed with the first four words of the key schedule.

$$\begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix} \oplus \begin{bmatrix} k_0 & k_4 & k_8 & k_{12} \\ k_1 & k_5 & k_9 & k_{13} \\ k_2 & k_6 & k_{10} & k_{14} \\ k_3 & k_7 & k_{11} & k_{15} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

B. ROUND PROCESSING

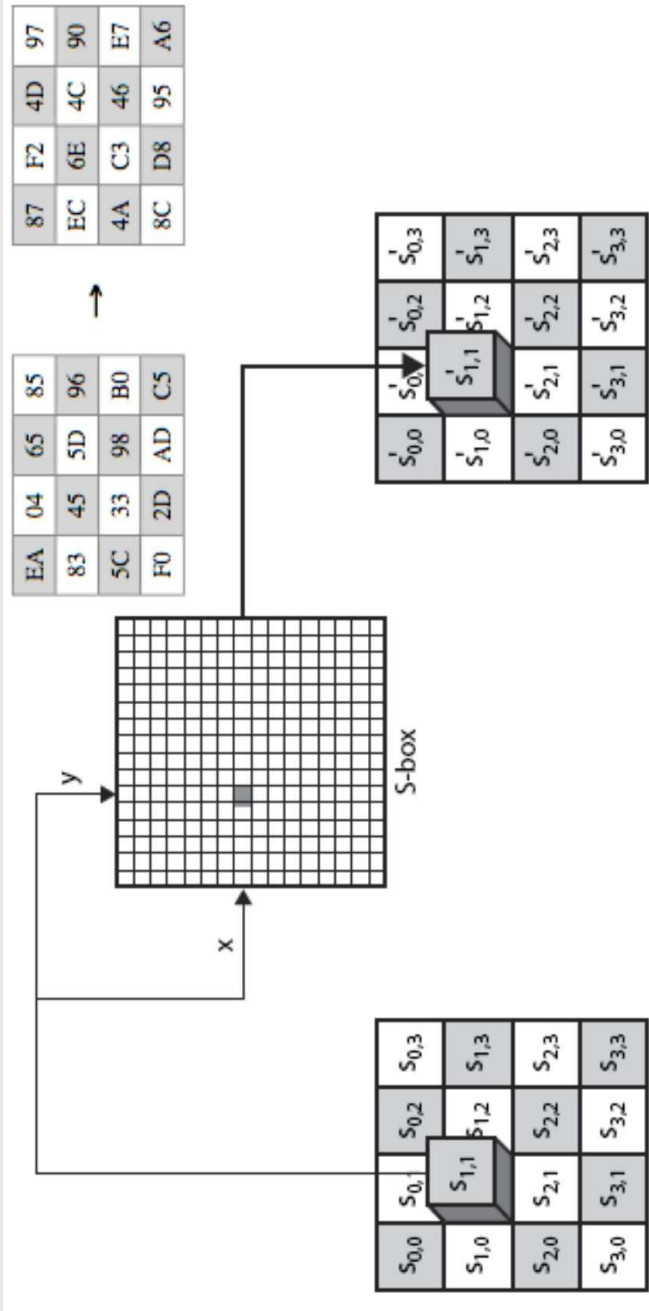
- The figure is showing another view of a single round of AES, emphasizing the mechanisms and **inputs** of each transformation.
- For encryption, each round consists of the following four steps:
 - 1) Substitute bytes,
 - 2) Shift rows,
 - 3) Mix columns, and
 - 4) Add round key.



B. ROUND PROCESSING

1. Substitute Bytes

- Each byte of the input state array is replaced by byte indexed by row (left 4 bits) & column (right 4-bits) of a 16x16 lookup table bits.
- The entries in the lookup table are created by using the notions of multiplicative inverses in GF (2⁸)



S-Box

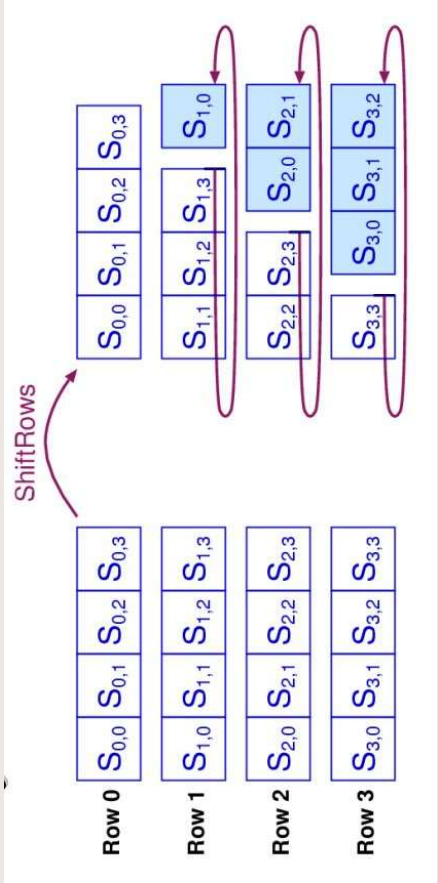
AES S-Box: Substitution values in hexadecimal notation for input byte (xy)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

B. ROUND PROCESSING

2. Shift Rows

- 1st row is unchanged row is unchanged
- 2nd row does 1 byte circular shift to left
- 3rd row does 2 byte circular shift to left
- 4th row does 3 byte circular shift to left



87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

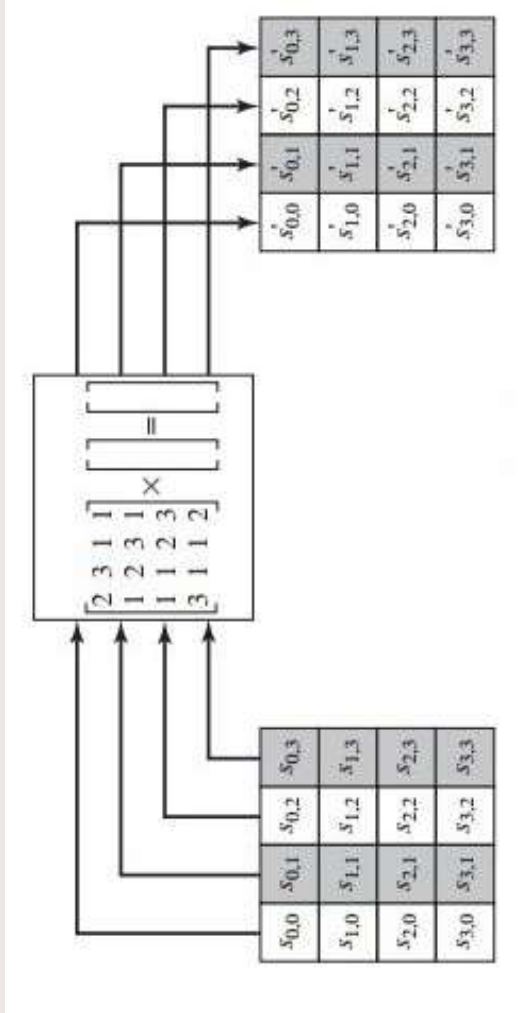
→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

B. ROUND PROCESSING

3. Mix Columns

- Effectively a matrix multiplication in $GF(2_8)$ using prime polynomial $m(x) = x^8+x^4+x^3+x+1$



$$s'_{0,j} = (2 \bullet s_{0,j}) \oplus (3 \bullet s_{1,j}) \oplus s_{2,j} \oplus s_{3,j}$$

$$s'_{1,j} = s_{0,j} \oplus (2 \bullet s_{1,j}) \oplus (3 \bullet s_{2,j}) \oplus s_{3,j}$$

$$s'_{2,j} = s_{0,j} \oplus s_{1,j} \oplus (2 \bullet s_{2,j}) \oplus (3 \bullet s_{3,j})$$

$$s'_{3,j} = (3 \bullet s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \bullet s_{3,j})$$

The following is an example of MixColumns:

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

→

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

B. ROUND PROCESSING

3. Mix Columns

$$\begin{array}{llll} (\{02\} \cdot \{87\}) \oplus (\{03\} \cdot \{6E\}) \oplus \{46\} & \oplus \{A6\} & = \{47\} \\ \{87\} & \oplus (\{02\} \cdot \{6E\}) \oplus (\{03\} \cdot \{46\}) \oplus \{A6\} & = \{37\} \\ \{87\} & \oplus \{6E\} & \oplus (\{02\} \cdot \{46\}) \oplus (\{03\} \cdot \{A6\}) = \{94\} \\ (\{03\} \cdot \{87\}) \oplus \{6E\} & \oplus \{46\} & \oplus (\{02\} \cdot \{A6\}) = \{ED\} \end{array}$$

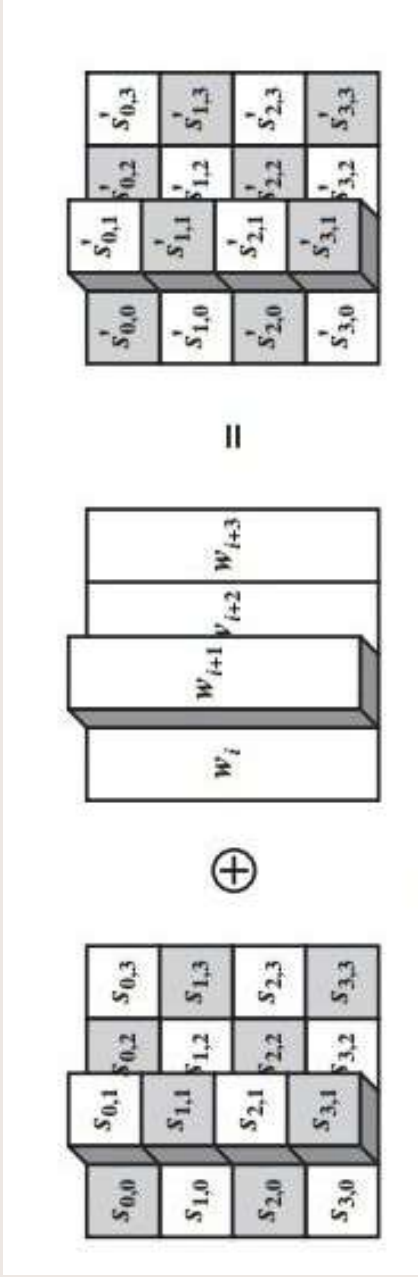
For the first equation, we have $\{02\} \cdot \{87\} = (0000\ 1110) \oplus (0001\ 1011) = (0001\ 0101)$ and $\{03\} \cdot \{6E\} = \{6E\} \oplus (\{02\} \cdot \{6E\}) = (0110\ 1110) \oplus (1101\ 1100) = (1011\ 0010)$. Then,

$$\begin{array}{ll} \{02\} \cdot \{87\} & = 0001\ 0101 \\ \{03\} \cdot \{6E\} & = 1011\ 0010 \\ \{46\} & = 0100\ 0110 \\ \{A6\} & = \underline{1010\ 0110} \\ & 0100\ 0111 = \{47\} \end{array}$$

B. ROUND PROCESSING

4. Add Round Key

- the 128 bits of input state are bitwise XORed with the 128 bits of the round key (key expansion is explained in the next slide)



47	40	A3	4C	AC	19	28	57	EB	59	8B	1B
37	D4	70	9F	77	FA	D1	5C	40	2E	A1	C3
94	E4	3A	42	66	DC	29	00	F2	38	13	42
ED	A5	A6	BC	F3	21	41	6A	1E	84	E7	D6

- The first matrix is State, and the second matrix is the round key

Key Expansion Algorithm

- First subkey (w_0, w_1, w_2, w_3) = cipher key
- Other words are calculated as follows:
 $w_i = w_{i-1} \oplus w_{i-4}$ for all values of i that are not multiples of 4.
- For the words with indices that are a multiple of 4 (w_{4k})
 $w_{4k} = g(w) \oplus w_{4k-4}$
 $g(w) = g(w_{4k-1})$
 $= \text{SubWord}(\text{RotWord}(w_{4k-1})) \oplus \text{Rcon}[k]$

1. RotWord performs a one-byte circular left shift on a word
2. SubWord performs a byte substitution on each byte of its input word, using the S-box (see S-box table in previous slide)
3. The result of steps 1 and 2 is XORed with a round constant, Rcon[j].

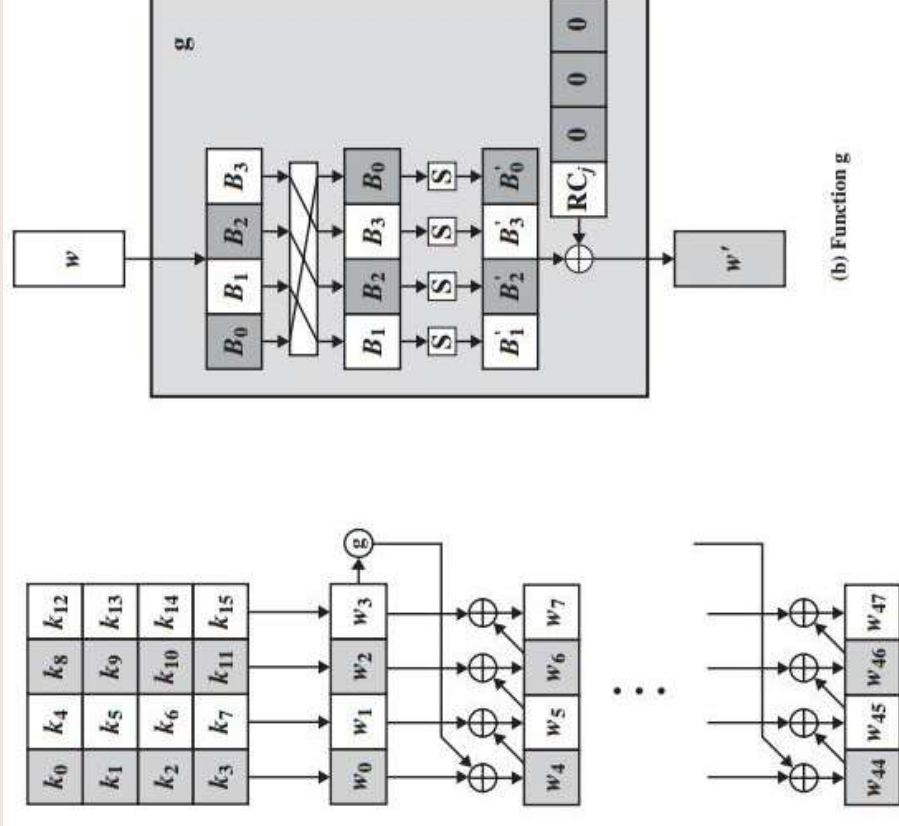


Figure 5.9 AES Key Expansion

Key Expansion Algorithm

- The round constant is different for each round and is defined as $Rcon[j] = (RC[j], 0, 0, 0)$, with $RC[1] = 1$, $RC[j] = 2 \cdot RC[j - 1]$ and with multiplication defined over the field $GF(2^8)$.
- The values of $RC[j]$ in hexadecimal are

j	1	2	3	4	5	6	7	8	9	10
$RC[j]$	01	02	04	08	10	20	40	80	1B	36

- **For example**, suppose that the round key for round 8 is
EA D2 73 21 B5 8D BA D2 31 2B F5 60 **7F 8D 29 2F**
- Then the first 4 bytes (first column) of the round key for round 9 are calculated as follows:

i (decimal)	temp	After RotWord	After SubWord	Rcon (9)	After XOR with Rcon	$w[i-4]$	$w[i] = temp \oplus w[i-4]$
36	7F8D292F	8D292F7F	5DA515D2	1B000000	46A515D2	EAD27321	AC7766F3

AES Example - Input (128 bit key and message)

AES Example - The first Roundkey

- Key in Hex (128 bits): 54 68 61 74 73 20 6D 79 20 4B 75 6E 67 20 46 75
- $w[0] = (54, 68, 61, 74)$, $w[1] = (73, 20, 6D, 79)$, $w[2] = (20, 4B, 75, 6E)$, $w[3] = (67, 20, 46, 75)$
- $g(w[3])$:
 - circular byte left shift of $w[3]$: $(20, 46, 75, 67)$
 - Byte Substitution (S-Box): $(B7, 5A, 9D, 85)$
 - Adding round constant $(01, 00, 00, 00)$ gives: $g(w[3]) = (B6, 5A, 9D, 85)$
- $w[4] = w[0] \oplus g(w[3]) = (E2, 32, FC, F1)$:

0101 0100	0110 1000	0110 0001	0111 0100
1011 0110	0101 1010	1001 1101	1000 0101
1110 0010	0011 0010	1111 1100	1111 0001
E2	32	FC	F1

- $w[5] = w[4] \oplus w[1] = (91, 12, 91, 88)$, $w[6] = w[5] \oplus w[2] = (B1, 59, E4, E6)$,
 $w[7] = w[6] \oplus w[3] = (D6, 79, A2, 93)$
- first roundkey: E2 32 FC F1 91 12 91 88 B1 59 E4 E6 D6 79 A2 93

AES Example - Add Roundkey, Round 0

State Matrix and Roundkey No.0 Matrix:

$$\begin{pmatrix} 54 & 4F & 4E & 20 \\ 77 & 6E & 69 & 54 \\ 6F & 65 & 6E & 77 \\ 20 & 20 & 65 & 6F \end{pmatrix}$$

$$\begin{pmatrix} 54 & 73 & 20 & 67 \\ 68 & 20 & 4B & 20 \\ 61 & 6D & 75 & 46 \\ 74 & 79 & 6E & 75 \end{pmatrix}$$

XOR the corresponding entries, e.g., $69 \oplus 4B = 22$

0110 1001

0100 1011

0010 0010

the new State Matrix is

$$\begin{pmatrix} 00 & 3C & 6E & 47 \\ 1F & 4E & 22 & 74 \\ 0E & 08 & 1B & 31 \\ 54 & 59 & 0B & 1A \end{pmatrix}$$

AES Example - Input (128 bit key and message)

Key in English: That's my Kung Fu (16 ASCII characters, 1 byte each)

Translation into Hex:

T	h	a	t	s	m	y	K	u	n	g	F	u			
54	68	61	74	73	20	6D	79	20	4B	75	6E	67	20	46	75

Key in Hex (128 bits): 54 68 61 74 73 20 6D 79 20 4B 75 6E 67 20 46 75

Plaintext in English: Two One Nine Two (16 ASCII characters, 1 byte each)

Translation into Hex:

T	w	o	O	n	e	N	i	n	e	T	w	o			
54	77	6F	20	4F	6E	65	20	4E	69	6E	65	20	54	77	6F

Plaintext in Hex (128 bits): 54 77 6F 20 4F 6E 65 20 4E 69 6E 65 20 54 77 6F

AES Example - All RoundKeys

- Round 0: 54 68 61 74 73 20 6D 79 20 4B 75 6E 67 20 46 75
- Round 1: E2 32 FC F1 91 12 91 88 B1 59 E4 E6 D6 79 A2 93
- Round 2: 56 08 20 07 C7 1A B1 8F 76 43 55 69 A0 3A F7 FA
- Round 3: D2 60 0D E7 15 7A BC 68 63 39 E9 01 C3 03 1E FB
- Round 4: A1 12 02 C9 B4 68 BE A1 D7 51 57 A0 14 52 49 5B
- Round 5: B1 29 3B 33 05 41 85 92 D2 10 D2 32 C6 42 9B 69
- Round 6: BD 3D C2 B7 B8 7C 47 15 6A 6C 95 27 AC 2E 0E 4E
- Round 7: CC 96 ED 16 74 EA AA 03 1E 86 3F 24 B2 A8 31 6A
- Round 8: 8E 51 EF 21 FA BB 45 22 E4 3D 7A 06 56 95 4B 6C
- Round 9: BF E2 BF 90 45 59 FA B2 A1 64 80 B4 F7 F1 CB D8
- Round 10: 28 FD DE F8 6D A4 24 4A CC C0 A4 FE 3B 31 6F 26

Round 1

AES Example - Round 1, Shift Row

- the current State Matrix is
$$\begin{pmatrix} 63 & EB & 9F & A0 \\ C0 & 2F & 93 & 92 \\ AB & 30 & AF & C7 \\ 20 & CB & 2B & A2 \end{pmatrix}$$
- four rows are shifted cyclically to the left by offsets of 0,1,2, and 3
- the new State Matrix is
$$\begin{pmatrix} 63 & EB & 9F & A0 \\ 2F & 93 & 92 & C0 \\ AF & C7 & AB & 30 \\ A2 & 20 & CB & 2B \end{pmatrix}$$
- this linear mixing step causes diffusion of the bits over multiple rounds

AES Example - Round 1, Mix Column

- Mix Column multiplies fixed matrix against current State Matrix:
$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} 63 & EB & 9F & A0 \\ 2F & 93 & 92 & C0 \\ AF & C7 & AB & 30 \\ A2 & 20 & CB & 2B \end{pmatrix} = \begin{pmatrix} BA & 84 & E8 & 1B \\ 75 & A4 & 8D & 40 \\ F4 & 8D & 06 & 7D \\ 7A & 32 & 0E & 5D \end{pmatrix}$$
- entry BA is result of $(02 \bullet 63) \oplus (03 \bullet 2F) \oplus (01 \bullet AF) \oplus (01 \bullet A2)$:
 - $02 \bullet 63 = 00000010 \bullet 01100011 = 11000110$
 - $03 \bullet 2F = (02 \bullet 2F) \oplus 2F = (00000010 \bullet 00101111) \oplus 00101111 = 01110001$
 - $01 \bullet AF = AF = 10101111$ and $01 \bullet A2 = A2 = 10100010$
 - hence
$$\begin{matrix} 11000110 \\ 01110001 \\ 10101111 \\ 10100010 \\ \hline 10111010 \end{matrix}$$

AES Example - Round 1, Substitution Bytes

- current State Matrix is
$$\begin{pmatrix} 00 & 3C & 6E & 47 \\ 1F & 4E & 22 & 74 \\ 0E & 08 & 1B & 31 \\ 54 & 59 & 0B & 1A \end{pmatrix}$$
- substitute each entry (byte) of current state matrix by corresponding entry in AES S-Box
- for instance: byte 6E is substituted by entry of S-Box in row 6 and column E, i.e., by 9F
- this leads to new State Matrix
$$\begin{pmatrix} 63 & EB & 9F & A0 \\ C0 & 2F & 93 & 92 \\ AB & 30 & AF & C7 \\ 20 & CB & 2B & A2 \end{pmatrix}$$
- this non-linear layer is for resistance to differential and linear cryptanalysis attacks

AES Example - Add Roundkey, Round 1

- State Matrix and Roundkey No.1 Matrix:
$$\begin{pmatrix} BA & 84 & E8 & 1B \\ 75 & A4 & 8D & 40 \\ F4 & 8D & 06 & 7D \\ 7A & 32 & 0E & 5D \end{pmatrix} \begin{pmatrix} E2 & 91 & B1 & D6 \\ 32 & 12 & 59 & 79 \\ FC & 91 & E4 & A2 \\ F1 & 88 & E6 & 93 \end{pmatrix}$$
- XOR yields new State Matrix
$$\begin{pmatrix} 58 & 15 & 59 & CD \\ 47 & B6 & D4 & 39 \\ 08 & 1C & E2 & DF \\ 8B & BA & E8 & CE \end{pmatrix}$$
- AES output after Round 1: 58 47 08 8B 15 B6 1C BA 59 D4 E2 E8 CD 39 DF CE

Rounds 2-7

AES Example - Round 2

- after Substitute Byte and after Shift Rows:

$$\begin{pmatrix} 6A & 59 & CB & BD \\ A0 & 4E & 48 & 12 \\ 30 & 9C & 98 & 9E \\ 3D & F4 & 9B & 8B \end{pmatrix}$$

- after Mixcolumns and after Roundkey:

$$\begin{pmatrix} 15 & C9 & 7F & 9D \\ CE & 4D & 4B & C2 \\ 89 & 71 & BE & 88 \\ 65 & 47 & 97 & CD \end{pmatrix}$$

AES Example - Round 5

- after Substitute Byte and after Shift Rows:

$$\begin{pmatrix} C8 & 30 & F2 & 94 \\ 74 & B0 & C8 & 37 \\ D1 & 20 & DD & 50 \\ 55 & F8 & 66 & B7 \end{pmatrix}$$

- after Mixcolumns and after Roundkey:

$$\begin{pmatrix} 2A & 26 & 8F & E9 \\ 78 & 1E & 0C & 7A \\ 1B & A7 & 6F & 0A \\ 5B & 62 & 00 & 3F \end{pmatrix}$$

AES Example - Round 3

- after Substitute Byte and after Shift Rows:

$$\begin{pmatrix} 1A & AB & 01 & 27 \\ B4 & 5B & 30 & 41 \\ D3 & BA & E9 & D2 \\ AA & E8 & BB & 9A \end{pmatrix}$$

- after Mixcolumns and after Roundkey:

$$\begin{pmatrix} AA & 65 & FA & 88 \\ 16 & 0C & 05 & 3A \\ 3D & C1 & DE & 2A \\ B3 & 4B & 5A & 0A \end{pmatrix}$$

AES Example - Round 6

- after Substitute Byte and after Shift Rows:

$$\begin{pmatrix} 14 & 26 & 4C & 15 \\ D1 & CF & 9C & 07 \\ B7 & 93 & 7A & 81 \\ 45 & 8C & 23 & B1 \end{pmatrix}$$

- after Mixcolumns and after Roundkey:

$$\begin{pmatrix} A9 & 37 & AA & F2 \\ AE & D8 & 0C & 21 \\ E7 & 6C & B1 & 9C \\ F0 & FD & 67 & 3B \end{pmatrix}$$

AES Example - Round 4

- after Substitute Byte and after Shift Rows:

$$\begin{pmatrix} BC & 51 & EE & B3 \\ 38 & 38 & EB & 12 \\ 04 & FF & 9A & 18 \\ 20 & 26 & 39 & A1 \end{pmatrix}$$

- after Mixcolumns and after Roundkey:

$$\begin{pmatrix} 10 & BC & D3 & F3 \\ D8 & 94 & E0 & E0 \\ 53 & EA & 9E & 25 \\ 24 & 40 & 73 & 7B \end{pmatrix}$$

AES Example - Round 7

- after Substitute Byte and after Shift Rows:

$$\begin{pmatrix} FA & 73 & BA & 58 \\ DC & 49 & D0 & 76 \\ 3F & F1 & 36 & 4F \\ F5 & 9B & 09 & 9D \end{pmatrix}$$

- after Mixcolumns and after Roundkey:

$$\begin{pmatrix} 9F & 37 & 51 & 37 \\ AF & EC & 8C & FA \\ 63 & 39 & 04 & 66 \\ 4B & FB & B1 & D7 \end{pmatrix}$$

$$\begin{pmatrix} 53 & 43 & 4F & 85 \\ 39 & 06 & 0A & 52 \\ 8E & 93 & 3B & 57 \\ 5D & F8 & 95 & BD \end{pmatrix}$$

Rounds 8-10

AES Example - Round 8

- after Substitute Byte and after Shift Rows:
$$\begin{pmatrix} ED & 1A & 84 & 97 \\ 12 & 6F & 67 & 00 \\ 19 & DC & E25B \\ 4C & 41 & 2A7A \end{pmatrix}$$
- after Mixcolumns and after Roundkey:
$$\begin{pmatrix} E88A & 4B & F5 \\ 74 & 75 & EE & E6 \\ D31F & 75 & 58 \\ 55 & 8A & 0C & 38 \end{pmatrix}$$

AES Example - Round 9

- after Substitute Byte and after Shift Rows:
$$\begin{pmatrix} 33 & 51 & 79 & 0A \\ 3F & 8B & 66 & 8F \\ EB & BE & 76 & 7D \\ 92 & C2 & 67 & 20 \end{pmatrix}$$
- after Mixcolumns and after Roundkey:
$$\begin{pmatrix} B6 & E7 & 51 & 8C \\ 84 & 88 & 98 & CA \\ 34 & 60 & 66 & FB \\ E8 & D7 & 70 & 51 \end{pmatrix}$$

AES Example - Round 10

- after Substitute Byte and after Shift Rows:
$$\begin{pmatrix} 01 & 3A & 8C & 21 \\ 33 & 3E & B0 & E2 \\ 3D & B8 & 8E & 04 \\ BC & 4D & 1C & A7 \end{pmatrix}$$
- after Roundkey (Attention: no Mix columns in last round):
$$\begin{pmatrix} 29 & 57 & 40 & 1A \\ C3 & 14 & 22 & 02 \\ 50 & 20 & 99 & D7 \\ 5F & F6 & B3 & 3A \end{pmatrix}$$
- ciphertext: 29 C3 50 5F 57 14 20 F6 40 22 99 B3 1A 02 D7 3A

Example: Concept of weak Keys

- Concept of weak keys does not apply to AES. Assume key with all zeros;

Pre-round:	00000000	00000000	00000000	00000000
Round 01:	62636363	62636363	62636363	62636363
Round 02:	9B9898C9	F9FBFBAA	9B9898C9	F9FBFBAA
Round 03:	90973450	696CCFFA	F2F45733	0B0FAC99
...	...	...	...	...
Round 10:	B4EF5BCB	3E92E211	23E951CF	6F8F188E

- The words in pre-round and first rounds are same. After the second rounds, every words is different.

Plaintext:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
Cipher Key:	24	75	A2	B3	34	75	56	88	31	E2	12	00	13	AA	54	87		
Ciphertext:	63	2C	D4	5E	5D	56	ED	B5	62	04	01	A0	AA	9C	2D	8D		

Example: Avalanche Effect

- **Avalanche Effect:** Changing a single bit in the plaintext affects many bits in the ciphertext.

Plaintext 1:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
Plaintext 2:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	0 <u>1</u>
Ciphertext 1:	63	2C	D4	5E	5D	56	ED	B5	62	04	01	A0	AA	9C	2D	8D			
Ciphertext 2:	26	F3	9B	BC	A1	9C	0F	B7	C7	2E	7E	30	63	92	73	13			

- Changed one bit in the last byte of plaintext 2. The results shows the effect of diffusion and confusion.
- The property of **confusion** hides the relationship between the ciphertext and the key.
- The idea of **diffusion** is to hide the relationship between the ciphertext and the plain text.

Example: Cipher key with all 0s

- Showing the impact of using a cipher key in which all bits are 0s.

Plaintext:	00	04	12	14	12	04	12	00	00	00	13	11	08	23	19	19
Cipher Key:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
Ciphertext:	5A	6F	4B	67	57	B7	A5	D2	C4	30	91	ED	64	9A	42	72

AES Decryption

- AES decryption is not identical to encryption
- But each step has an inverse

AES Summary

- AES encrypts 128 bit blocks with 128 AES encrypts 128 bit blocks with 128-bit, 192 bit, 192-bit or 256 bit or 256-bit keys using 10, 12, or 14 rounds, respectively.
- Is not a Feistel cipher -> All 128 bits are encrypted 128 bits are encrypted
- Each round = 4 steps of SubBytes, ShiftRows, MixColumns, and AddRoundKey.
- Last round has only 3 steps. No MixColumns.
- Decryption is not the same as encryption (as in DES).
- Decryption consists of inverse steps.

More Question

- Given the plaintext [0001 0203 0405 0607 0809 0A0B 0C0D 0E0F] and the key [0101 0101 0101 0101 0101 0101 0101 0101]
 - a. Show the original contents of state, displayed as a 4x4 matrix.
 - b. Show the value of state after initial AddRoundKey.
 - c. Show the value of State after SubBytes.
 - d. Show the value of State after ShiftRows.
 - e. Show the value of State after MixColumns.

More Questions

- A block cipher breaks the input into fixed-size blocks and then progresses the crypto operations.
- a) True
- b) False
- _____ is a example of stream cipher
- a) Caesar cipher
- b) Vigenere cipher
- c) RC4
- d) Data Encryption Standard (DES)
- Explain the two different types of stream ciphers.
- How many rounds (iterations) each with a round key are present in the DES Algorithm ?
- What is linear feedback shift registers (LFSR)? For n-bit LFSR, what is the longest possible sequence?

More Questions

- What is the majority function in A5/1 stream cipher?
- What are the advantages of RC4.
- What will be the plain text for a cipher text [4 3 2 3] using RC4 with the key stream [5 1 0 1] ?
- How should multiple blocks be encrypted with a block cipher?
- Explain CBC (Cipher Block Chaining) mode with help of diagram.
- Explain the swap and circular shift encryption and decryption components of a Modern Block Cipher.
- Explain DES using a suitable block-diagram showing rounds, key and permutations.
- What is the Feistel Cipher? Explain its encryption and decryption functions.